

**FEDERAL UNIVERSITY OF TECHNOLOGY, OWERRI, IMO
STATE**

**SCHOOL OF INFORMATION AND COMMUNICATION
TECHNOLOGY**

DEPARTMENT OF INFORMATION TECHNOLOGY

**PERFORM+: PERFORMANCE TRACKING AND ADVISORY
SYSTEM**

BY

OKORO EMEKA JOHN

20211258122

A PROJECT PRESENTED TO THE DEPARTMENT OF INFORMATION
TECHNOLOGY, SCHOOL OF INFORMATION AND COMMUNICATION
TECHNOLOGY, FEDERAL UNIVERSITY OF TECHNOLOGY, OWERRI, IN
PARTIAL FULFILMENT FOR THE AWARD OF BACHELOR OF
TECHNOLOGY (B.TECH) IN INFORMATION TECHNOLOGY

2024/2025 SESSION

CERTIFICATION

I certify that this research "Perform+: Performance Tracking and Advisory System" was carried out by Okoro Emeka John (20211258132) in partial fulfilment for the award of the degree of Bachelor of Technology (B.Tech) in Information Technology, of the Federal University of Technology, Owerri.

(Project Supervisor)

Date

(Head of Department)

Date

(Dean, School of ICT)

Date

(External Examiner)

Date

DEDICATION

This project is dedicated to Almighty God, the giver of wisdom and knowledge, and to my parents, whose unwavering support, prayers, and sacrifice made this academic journey possible.

ACKNOWLEDGEMENT

I am deeply grateful to my project supervisor for their invaluable guidance, patience, and constructive criticism throughout the course of this research. I also wish to acknowledge the Head of Department and the entire academic staff of the Department of Information Technology, Federal University of Technology, Owerri, for the knowledge imparted to me throughout my programme of study.

My sincere appreciation also goes to my family and friends for their moral and financial support, and to my course mates for their encouragement during the course of this work. Finally, I thank every student whose frustration with manual CGPA calculation, repeated to me in conversation after conversation, was the real motivation behind this project.

ABSTRACT

The accurate and consistent computation of Cumulative Grade Point Average (CGPA) remains a persistent challenge for undergraduate students in Nigerian universities, many of whom rely on manual calculation or generic spreadsheet templates that are error-prone and fail to account for institution-specific grading policies such as the treatment of carryover (repeated) courses. This study presents the design, development, and evaluation of Perform+, a mobile application built with the Flutter framework for Android devices, intended to provide Federal University of Technology, Owerri (FUTO) students with an accurate, offline-first tool for tracking academic performance. The system was developed using an Agile, iterative methodology and implements a deterministic CGPA computation engine that correctly applies the National Universities Commission-aligned grading and repeat-policy rules governing Nigerian undergraduate programmes, ensuring that a student's classification is computed correctly regardless of how many courses have been repeated. Perform+ integrates a local SQLite database, accessed through the Drift persistence library, as its primary source of truth, enabling the application to function fully without an internet connection, while a Supabase/PostgreSQL backend provides optional cloud synchronisation, authentication, and row-level-secured data backup. The application further incorporates an artificial-intelligence-assisted academic advisory feature, in which a large language model is used strictly to phrase natural-language guidance around figures already computed by the deterministic engine, together with a result-slip scanning feature that extracts course information from a photographed registration slip to reduce manual data entry. The completed system was tested using unit tests against the CGPA engine and systematic exercising of the

user interface, with all test cases passing, including verification of three distinct carryover-policy outcomes on an identical transcript. The study concludes that a locally verifiable, policy-correct, offline-first approach to academic performance tracking, augmented rather than replaced by artificial intelligence, offers a meaningful improvement over the manual and semi-automated tools presently available to Nigerian university students.

TABLE OF CONTENTS

CHAPTER ONE: INTRODUCTION	1
1.1 Background of the Study	1
1.2 Statement of the Problem	2
1.3 Aim and Objectives of the Study	3
1.4 Research Questions	4
1.5 Scope of the Study	4
1.6 Limitations of the Study	5
1.7 Significance of the Study	5
1.8 Definition of Terms	6
1.9 Organization of the Work	7
CHAPTER TWO: LITERATURE REVIEW	8
2.1 Conceptual Framework	8
2.2 Theoretical Framework	9
2.3 Empirical Framework	11
2.4 Summary of Literature Review	13
CHAPTER THREE: RESEARCH METHODOLOGY	15
3.1 Methodology Adopted	15
3.2 System Analysis	16
3.3 Functional Requirements	17
3.4 Non-Functional Requirements	18
3.5 Tools and Technologies Used	19
3.6 Data Collection Method	20
3.7 System Design Methodology	21
3.8 Summary	21
CHAPTER FOUR: SYSTEM DESIGN AND IMPLEMENTATION	22
4.1 System Architecture	22
4.2 Database Design	24
4.3 Grading Scheme Representation	26
4.4 The CGPA Computation Engine	27
4.5 Key Modules	29
4.6 User Interface Implementation and Screens	30
4.7 Implementation Summary	49

4.8 Testing	50
4.9 Summary	51
CHAPTER FIVE: RESULTS AND DISCUSSION	52
5.1 Presentation of Results	52
5.2 Testing Results	53
5.3 Discussion of Findings	54
5.4 Summary	55
CHAPTER SIX: SUMMARY, CONCLUSION AND RECOMMENDATIONS ...	56
6.1 Summary of Findings	56
6.2 Conclusion	56
6.3 Recommendations	57
6.4 Future Work	58
BIBLIOGRAPHY	59
APPENDICES	62
Appendix A: CGPA Computation Engine Source Code	62
Appendix B: Sample Unit Test Cases	68
Appendix C: Repeat Policy — Worked Numerical Example	70

LIST OF TABLES

Table 1.1: Breakdown of the Project Report Structure	7
Table 2.1: Summary of Key Reviewed Literature	12
Table 3.1: Summary of Functional Requirements by Module	18
Table 4.1: Database Schema Summary	24
Table 4.2: Grading Scale and Grade Points (FUTO 5.0 Scale)	27
Table 4.3: Classification Bands (FUTO)	27
Table 4.4: Sample Test Cases	50
Table 5.1: Summary of Achieved Objectives	52
Table 5.2: System Usability and Performance Observations	53
Table 5.3: Comparison of Perform+ with Existing Mobile CGPA Applications ...	55
Table C.1: Repeat Policy — Worked Numerical Example	70

LIST OF FIGURES

Figure 2.1: Conceptual Framework of Academic Performance Tracking	9
Figure 3.1: Agile Iterative Development Cycle Adopted for This Study	16
Figure 4.1: System Architecture of Perform+	23
Figure 4.2: Entity-Relationship Diagram of the Perform+ Database	24
Figure 4.3: Data Flow Diagram (Level 0) of Perform+	25
Figure 4.4: Use Case Diagram for Perform+	26
Figure 4.5: Repeat-Course (Carryover) Policy Decision Flowchart	28
Figure 4.7: Institution and Grading Scheme Selection Screen	32
Figure 4.8: Add Course Results Screen	33
Figure 4.9: GPA Reveal Screen	35
Figure 4.10: Sign-Up / Sign-In Screen	36
Figure 4.11: Goal-Setting Screen	38
Figure 4.12: Dashboard (Home) Screen	40
Figure 4.13: Academics Tab — Results View	42
Figure 4.14: Academics Tab — Roadmap View	43
Figure 4.15: Academics Tab — Reports View	44
Figure 4.16: AI Advisor Screen	46
Figure 4.17: Study Tab Screen	48
Figure 4.18: Me (Profile) Tab Screen	49

CHAPTER ONE

INTRODUCTION

1.1 Background of the Study

The widespread adoption of smartphones among university students has fundamentally altered how academic information is accessed, recorded, and acted upon. Crompton, Burke and Gregory (2017) observe that smartphones have become the primary device through which university students access digital resources, a trend that makes mobile-first design an essential consideration for any application intended to support the modern undergraduate experience. Within this broader shift, the systematic analysis of student academic records to support decision-making has emerged as a distinct field of study. Romero and Ventura (2010) describe educational data mining as the process of extracting useful patterns from academic datasets to improve learning outcomes and institutional decision-making, while Siemens (2013) defines the closely related field of learning analytics as the measurement, collection, analysis, and reporting of data about learners and their contexts, with the purpose of understanding and optimising learning. Ferguson (2012) further notes that the growth of learning analytics has been driven by the increasing availability of educational data, advances in data science, and institutional pressure to improve student retention.

In Nigerian universities, the Cumulative Grade Point Average (CGPA) is the single most consequential academic metric a student accumulates: it determines eligibility for honours classifications, postgraduate admission, and, in many cases, employment opportunities. Despite its importance, the computation of CGPA at many Nigerian institutions, including the Federal University of Technology, Owerri (FUTO), remains

a largely manual exercise. Students typically rely on hand calculation, generic spreadsheet templates, or informal mobile applications that were not designed around the specific grading scale, classification bands, and repeat-course (carryover) policy of their institution. This creates a gap between the sophistication of modern mobile technology and the rudimentary tools still in everyday use for one of the most important calculations in a student's academic life.

The problem is compounded by the fact that a Nigerian undergraduate's transcript is rarely simple: by the final year, a typical student has accumulated results across eight to twelve semesters, several hundred credit units, and, for a meaningful proportion of students, at least one carryover (a course failed and later repeated). Each of these carryover courses interacts with institutional policy in a way that a generic calculator cannot be expected to know. Perform+ was conceived to close this gap: a purpose-built, offline-capable mobile application that computes CGPA correctly against an institution's actual grading scheme, tracks a student's standing over time, and offers academic guidance grounded in those same verified figures.

1.2 Statement of the Problem

Existing mobile solutions for CGPA-related computation in the Nigerian context suffer from several recurring limitations. Khamis, Khalil, Al-Maaitah and Al-Maaitah (2023) report that mobile applications for CGPA calculation have achieved strong student adoption, but observe that such tools generally lack integration with advisory or predictive features, offering a bare numeric output with no guidance on what a student should do next. Similarly, Nosseir and Fathy (2020) found that mobile applications designed specifically for tracking university student performance improve academic self-awareness and enable timely intervention, yet many available

implementations are built as generic calculators rather than as complete academic-management tools. A further, Nigeria-specific constraint is infrastructural: Oyefara, Adejoh, Adisa, Abdulsalam and Alabi (2022) document that Nigerian university stakeholders face significant barriers to technology adoption, including inadequate infrastructure, unreliable internet connectivity, and insufficient institutional support, a reality that renders any tool requiring constant internet access impractical for everyday use on Nigerian campuses.

Beyond these general limitations, no widely available mobile application correctly implements the specific repeat-course (carryover) policy mandated for Nigerian undergraduate programmes. A single transcript, computed under different repeat-policy assumptions, can legitimately produce markedly different CGPA outcomes, meaning that a tool which gets this calculation wrong does not merely display an inconvenient number; it can misinform a student about their actual academic standing and classification. A student who is told, incorrectly, that a First Class average is within reach may make course-load and registration decisions around that false belief; a student who is told, incorrectly, that it is not, may give up on an attainable goal. This problem, combined with the absence of offline support and the absence of any advisory layer built on top of a student's real data, defines the specific gap that this study addresses.

1.3 Aim and Objectives of the Study

The aim of this study is to design, develop, and evaluate Perform+, a mobile application that enables Federal University of Technology, Owerri students to accurately compute, track, and project their academic performance, with guidance generated from their own verified results.

The specific objectives of the study are to:

1. design and implement a deterministic CGPA computation engine that correctly applies grading-scheme rules and repeat-course (carryover) policy variants consistent with Nigerian university regulation;
2. design and implement an offline-first local data storage layer that allows the application to remain fully functional without internet connectivity, while supporting optional cloud synchronisation for data durability;
3. design and implement a result-slip scanning feature that extracts course information from a photographed academic document to reduce the burden of manual data entry;
4. design and implement an artificial-intelligence-assisted academic advisory feature that generates natural-language guidance strictly from figures already computed by the deterministic engine; and
5. evaluate the completed system through unit and systematic interface testing to verify the correctness of its computations and the functionality of its user interface.

1.4 Research Questions

The study is guided by the following research questions:

6. How can a mobile application be designed to compute CGPA in a manner that remains correct across the different repeat-course policy variants used by Nigerian universities?
7. To what extent can an offline-first local storage architecture provide uninterrupted academic-tracking functionality under the infrastructural constraints typical of Nigerian university campuses?

8. How can optical scanning of a registration slip be used to reduce the manual data-entry burden placed on students?
9. How can a large language model be incorporated into an academic advisory feature without compromising the factual accuracy of the figures it presents to students?

1.5 Scope of the Study

This study covers the design, implementation, and testing of Perform+ as a native Android mobile application built with the Flutter framework. The scope includes the CGPA and GPA computation engine, grading-scheme and classification-band handling, local and cloud data persistence, user authentication, result-slip scanning, dashboard analytics, goal projection, and the artificial-intelligence-assisted advisory feature. The grading scheme implemented and verified in depth is that of the Federal University of Technology, Owerri; the application's data model is nonetheless designed to accommodate other Nigerian institutions' grading schemes in principle, and sixteen institutions are seeded into the institution picker for this reason. The study does not cover an iOS release, payment or subscription functionality, or integration with any official university student-information-system portal, all of which are identified as future work in Chapter Six.

1.6 Limitations of the Study

The study is subject to the following limitations. First, the application does not integrate directly with FUTO's official student portal; results must be entered by the student, either manually or via the result-slip scanning feature, rather than retrieved automatically from an institutional database. Second, while the application's data model supports multiple institutions, only the FUTO grading scheme was verified in

depth against the institution's own academic regulations during this study; grading schemes seeded for other institutions should be treated as unverified defaults. Third, two features — cloud synchronisation and the artificial-intelligence-assisted advisory feature — require internet connectivity to operate, although the core CGPA-tracking functionality of the application remains fully available offline. Fourth, the study was conducted and tested within a limited timeframe and on a limited set of devices, and so performance across the full diversity of Android hardware in use by Nigerian students was not exhaustively verified.

1.7 Significance of the Study

This study is significant to several groups of stakeholders. For students, Perform+ offers an accurate, always-available tool for tracking academic standing and planning toward a target classification, reducing reliance on error-prone manual computation. For academic advisers and departments, the study demonstrates how learning-analytics principles can be applied at the level of an individual student's mobile device rather than only within institutional systems; Susnjak, Ramaswami and Mathrani (2022) note that learning-analytics dashboards which surface actionable insight have been shown to significantly improve student engagement and academic self-regulation, a benefit this study extends to a personal, offline-capable context. For the wider body of software-engineering and educational-technology research in Nigeria, the study contributes a concrete, tested example of how a deterministic computation engine and a large language model can be combined responsibly within an academic tool, with the language model restricted to explanation rather than calculation.

1.8 Definition of Terms

CGPA (Cumulative Grade Point Average): the credit-unit-weighted average of grade points earned across every semester on a student's record.

GPA (Grade Point Average): the credit-unit-weighted average of grade points earned within a single semester.

Carryover: a course failed in one semester and registered again in a later semester; also called a repeated or retaken course.

Repeat Policy: the institution-specific rule governing how a carryover course's grade is counted toward CGPA.

Classification: the honours category (e.g. First Class, Second Class Upper) derived from a student's final CGPA against an institution's classification bands.

Grading Scheme: the combination of score-to-letter mapping, letter-to-point mapping, classification bands, and repeat policy used by a specific institution.

Offline-first: an architecture in which an application's core functionality works fully without a network connection, treating connectivity as an enhancement rather than a requirement.

1.9 Organization of the Work

The remainder of this report is organized into five further chapters. Chapter Two reviews literature relevant to learning analytics, mobile applications for academic performance tracking, artificial intelligence in education, and cross-platform mobile development, and identifies the gap this study addresses. Chapter Three describes the research methodology adopted, including the system-development approach, requirements specification, and tools used. Chapter Four presents the system design and implementation, covering the application's architecture, database design, CGPA

computation engine, and key feature modules, illustrated with screenshots of the completed system. Chapter Five presents the results obtained, the outcomes of system testing, and a discussion of findings in relation to the literature reviewed in Chapter Two. Chapter Six concludes the study with a summary of findings, conclusions, recommendations, and suggestions for future work, followed by the list of references and appendices.

Table 1.1: Breakdown of the Project Report Structure

Chapter	Description
Chapter One	Introduction
Chapter Two	Literature Review
Chapter Three	Research Methodology
Chapter Four	System Design and Implementation
Chapter Five	Results and Discussion
Chapter Six	Summary, Conclusion and Recommendations

Source: Researcher, 2025

CHAPTER TWO

LITERATURE REVIEW

2.1 Conceptual Framework

This section introduces the core concepts underlying the study: academic performance tracking, educational data mining, learning analytics, and mobile learning. Academic performance tracking refers to the systematic recording and analysis of a student's grades over time in order to derive an overall standing, typically expressed, in the Nigerian context, as a Cumulative Grade Point Average (CGPA). The analytical foundation for this activity is provided by educational data mining, which Romero and Ventura (2010) describe as the process of extracting useful patterns from academic datasets to improve learning outcomes and institutional decision-making. Closely related is learning analytics, defined by Siemens (2013) as the measurement, collection, analysis, and reporting of data about learners and their contexts, with the purpose of understanding and optimising learning. Ferguson (2012) situates the growth of learning analytics within three converging drivers: the increasing availability of educational data, advances in data science techniques capable of processing that data, and institutional pressure to improve student retention and outcomes. Perform+ operates at the intersection of these two fields, applying learning-analytics principles not at the institutional level, where they are traditionally deployed, but at the level of an individual student's own device.

The third foundational concept is mobile learning, understood here broadly as the use of mobile devices to support activities related to a student's academic life, including but not limited to formal instruction. Crompton Etol (2017) observe that smartphones

have become the primary device through which university students access digital resources, a finding that establishes mobile-first design as a reasonable default for any contemporary educational tool. However, Kaliisa and Michelle (2019) caution that mobile learning adoption across African universities is constrained by infrastructure deficits, policy gaps, and device accessibility disparities, meaning that a mobile application intended for an African university context cannot assume the same constant connectivity that may be assumed elsewhere. This conceptual tension — mobile-first design on one hand, and infrastructural unreliability on the other — is precisely what motivates the offline-first architectural decision examined in Chapter Four.

Figure 2.1 summarises how these concepts relate to one another within the scope of this study: raw academic records are the input; educational data mining and learning analytics are the analytical lens applied to them; mobile learning is the delivery context; and the output is a student-facing academic performance tracking tool.

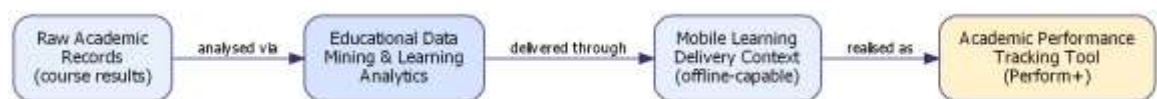


Figure 2.1: Conceptual Framework of Academic Performance Tracking

2.2 Theoretical Framework

The theoretical framework of this study draws on the body of research examining the application of artificial intelligence, and specifically large language models (LLMs), within educational contexts. Zawacki-Richter, Marín, Bond and Gouverneur (2019) conducted a systematic review of artificial intelligence applications in higher education and found that most existing applications concentrate on intelligent tutoring, adaptive learning, and automated assessment, with comparatively limited

attention paid to personal academic performance tracking — a gap this study's advisory feature is positioned to help address. Since that review was published, the emergence of large, general-purpose language models has substantially widened what is theoretically possible. Kasneci et al (2023) argue that while large language models present significant opportunities for personalised learning support, they simultaneously raise serious concerns around factual accuracy and the risk of students over-relying on model-generated output without independent verification. Wang et al (2023) similarly characterise large language models as holding transformative potential in education, citing intelligent feedback generation, content generation, and personalised academic support as key capability areas, while cautioning that this potential is only realised when the technology is deployed with appropriate safeguards.

The specific safeguard most relevant to this study is the theoretical separation between computation and explanation. Yan et al (2024), in a systematic scoping review of the practical and ethical challenges of large language models in education, identify misinformation, lack of transparency, and data-privacy risk as critical, recurring challenges in deploying such models within educational environments — misinformation being the most directly relevant risk for an application whose core purpose is reporting a student's academic standing. One documented mitigation strategy against this risk is retrieval-augmented generation (RAG), in which a language model's output is grounded in externally supplied, verified data rather than left to the model's own internal, sometimes unreliable, recall. Liu et al (2025) demonstrate that LLM-based tutoring systems augmented with retrieval-augmented generation achieve improved factual accuracy and personalisation in the academic feedback they generate, compared with ungrounded prompting. Perform+ adopts a

theoretically stronger variant of this same principle: rather than retrieving supporting data at generation time, the deterministic CGPA engine computes every figure in advance, and the language model is given no arithmetic task at all — it is restricted purely to phrasing natural-language guidance around numbers that have already been calculated and verified. This design is the direct theoretical consequence of the risks identified by Kasneci et al (2023) and Yan et al (2024): if an LLM cannot be trusted to compute reliably, the architecture must simply never ask it to.

A second theoretical strand relevant to this study is self-regulated learning theory, which holds that learners who can monitor, evaluate, and adjust their own progress against a goal perform better than those who cannot. Susnjak et al (2022) connect this theory directly to learning-analytics dashboard design, arguing that a dashboard which surfaces actionable insight — not merely raw numbers — is what enables a learner to actually regulate their own behaviour in response to it. This theoretical position directly informs the goal-setting and projection features described in Chapter Four: a target classification and the pace required to reach it are only useful to a student if presented as something actionable, not as an abstract statistic.

2.3 Empirical Framework

Empirical work specific to mobile CGPA and academic-performance-tracking applications provides the most direct precedent for this study. Nosseir and Fathy (2020) developed and evaluated a performance-management mobile application for university students and found that applications of this kind improve academic self-awareness and enable more timely intervention when a student's performance begins to decline. Khamis et al (2023) report on a mobile application for CGPA calculation and find that such tools achieve strong student adoption, but observe that the

applications studied generally lack any integration with advisory or predictive features — they compute a number and stop there. This is the specific empirical gap that Perform+'s advisory feature is designed to close: coupling a correctly computed figure with natural-language guidance about what that figure means and what the student might do next.

A second strand of empirical work examines the prediction of student outcomes from behavioural and academic data. Conijn, Snijders, Kleingeld and Matzat (2017) show that student performance can be reliably predicted from learning-management-system activity data, enabling early identification of at-risk students before formal examinations take place. In a Nigerian setting specifically, Ekubo and Esiefarienrhe (2022) demonstrate that machine-learning models can predict low academic performance with up to 98% accuracy using historical student records drawn from a Nigerian university. Perform+ does not implement predictive machine learning of this kind — consistent with the project's explicit rule against fabricating probability scores from a single transcript with no cohort baseline — but this body of work establishes that structured academic data of the kind the application already collects carries substantial, empirically demonstrated analytical value, and identifies real institutional grade-distribution data as the necessary precondition before such a feature could be responsibly added.

Finally, Oyefara Etol (2022) provide empirical grounding for the infrastructural constraints under which any Nigerian academic-technology tool must operate, documenting the barriers faced by Nigerian university stakeholders in adopting technology, including inadequate infrastructure, unreliable internet connectivity, and insufficient institutional support. This finding is taken directly as a design constraint

in this study: Chapter Four's discussion of the offline-first local database architecture is a direct response to the empirical reality that Oyefara Etol (2022) document.

Table 2.1: Summary of Key Reviewed Literature

Author(s) & Year	Focus	Key Finding	Relevance to This Study
Romero & Ventura (2010)	Educational data mining	Defines EDM as pattern extraction from academic data	Analytical foundation for the engine
Siemens (2013)	Learning analytics	Defines LA as measurement/analysis of learner data	Theoretical basis for dashboard design
Ferguson (2012)	LA drivers	Data availability, data science, retention pressure	Motivates data-driven tracking
Crompton Etol (2017)	Smartphones in HE	Smartphones are the primary access device	Justifies mobile-first delivery
Kaliisa & Michelle (2019)	Mobile learning in Africa	Infrastructure/policy gaps constrain adoption	Justifies offline-first design
Nosseir & Fathy (2020)	Mobile performance app	Improves self-awareness, timely intervention	Direct precedent for this study
Khamis Etol (2023)	Mobile CGPA app	Strong adoption, lacks advisory features	Identifies the advisory-feature gap
Zawacki-Richter Etol (2019)	AI in higher education	Most AI tools focus on tutoring, not tracking	Positions the advisory feature
Kasneci etal (2023)	LLMs in education	Opportunities and accuracy risks of LLMs	Justifies grounding the advisor
Wang etal (2023)	LLM potential in education	Transformative potential, needs safeguards	Supports the AI advisory design
Yan etal (2024)	LLM ethical challenges	Misinformation, transparency, privacy risks	Core justification for the engine/AI split
Liu etal (2025)	RAG-based tutoring	RAG improves factual accuracy	Precedent for grounding model output
Conijn Etol (2017)	Predicting performance from LMS data	Reliable early prediction is possible	Frames future predictive work
Ekubo & Esiefarienrhe (2022)	ML prediction, Nigeria	98% accuracy predicting low performance	Nigerian empirical precedent
Oyefara Etol (2022)	ICT barriers, Nigeria	Infrastructure/connectivity barriers	Justifies offline-first architecture
Susnjak Etol (2022)	LA dashboards	Actionable dashboards improve self-regulation	Informs dashboard/goal design

Source: Researcher's Review of Literature, 2025

2.4 Summary of Literature Review

Taken together, the literature reviewed in this chapter establishes three things. First, the conceptual and theoretical foundations of learning analytics and educational data mining (Romero & Ventura, 2010; Ferguson, 2012; Siemens, 2013) justify treating academic performance data as a legitimate object of systematic analysis, not merely

administrative record-keeping. Second, the growing body of research on large language models in education (Zawacki-Richter Etol, 2019; Kasneci etal, 2023; Wang etal, 2023; Yan etal, 2024; Liu etal, 2025) establishes both the genuine potential and the well-documented risks of using such models in an academic tool, with the consistent theoretical conclusion that factual grounding is non-negotiable. Third, the empirical literature on mobile performance-tracking applications (Nosseir & Fathy, 2020; Khamis Etol, 2023) and on the Nigerian technology-adoption context specifically (Oyefara Etol, 2022; Ekubo & Esiefarienrhe, 2022; Kaliisa & Michelle, 2019) identifies a clear, unaddressed gap: no widely available tool combines a policy-correct, Nigeria-specific CGPA engine with an offline-first architecture and a factually grounded advisory layer. This gap is precisely what Perform+ is designed to fill, and the remaining chapters of this report describe how.

CHAPTER THREE

RESEARCH METHODOLOGY

3.1 Methodology Adopted

This study adopts the Agile, iterative software-development methodology. Under this methodology, the system is built in a sequence of short, functionally complete increments — in this project's case, one feature area at a time (the CGPA computation engine, followed by local persistence, result-slip scanning, cloud synchronisation, dashboard analytics, and finally the AI advisory feature) — rather than attempting to specify and build the entire system in a single, long pass before any part of it can be tested.

Agile was chosen over a traditional Waterfall approach for three reasons specific to this project. First, the single highest-risk component of the system is the CGPA computation engine's handling of repeat-course (carryover) policy, where a wrong assumption produces a plausible-looking but incorrect result; an iterative methodology allows this component to be built and exhaustively unit-tested in complete isolation, before any dependent feature (projection, dashboard analytics, AI advisory output) is built on top of it, so that a defect at this layer is caught immediately rather than discovered only after the whole system is assembled. Second, the requirements for an academic-tracking application are naturally incremental: a working CGPA calculator is independently useful before result-slip scanning exists, which is in turn independently useful before an AI advisory layer exists, allowing each increment to be validated on its own merits. Third, Agile's emphasis on continuous testing aligns directly with this project's own non-negotiable rule that the

engine must never be committed with a failing test, which is only practical to enforce if testing happens continuously throughout development rather than as a single phase at the end.

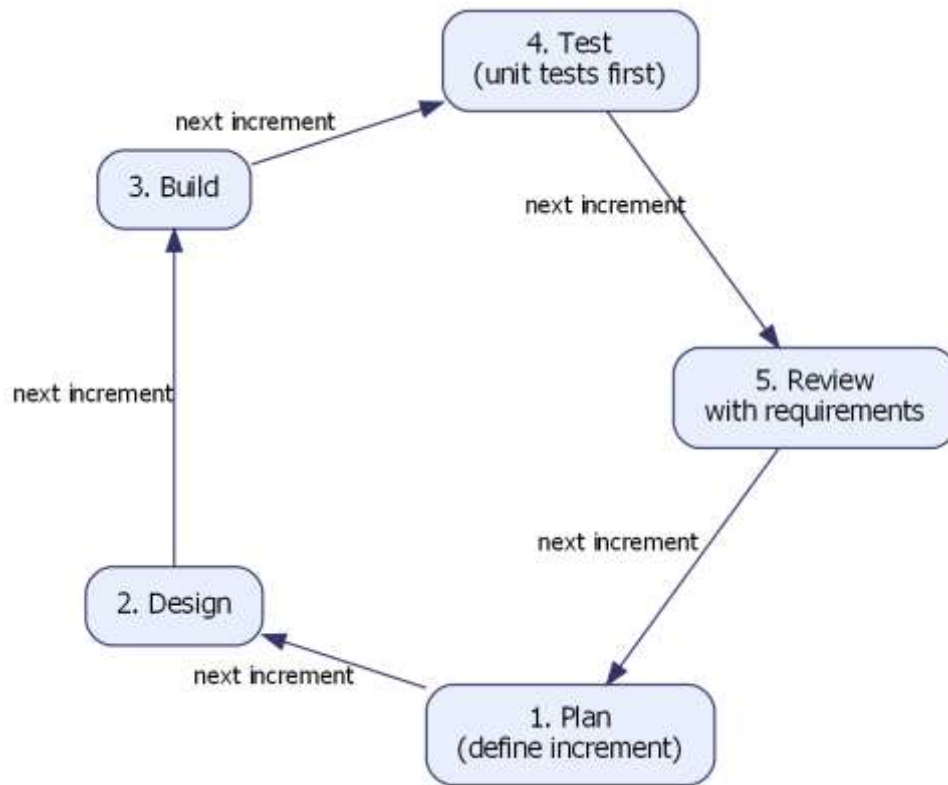


Figure 3.1: Agile Iterative Development Cycle Adopted for This Study

3.2 System Analysis

System analysis for this project began with an examination of the existing manual process Nigerian university students use to track their CGPA: hand calculation from a printed or photographed result slip, performed independently for each semester, with no persistent record of previous semesters' figures beyond whatever the student has kept on paper or in a spreadsheet. This manual process was analysed against the specific grading and repeat-course policy of the Federal University of Technology, Owerri, and found to be error-prone in exactly the way the literature predicts (Khamis Etol, 2023) — particularly around the correct treatment of a retaken course, where

three structurally different, individually plausible calculation approaches exist and produce three different numeric outcomes from an identical transcript. This analysis directly informed the system's core requirement: that CGPA computation must be centralised in one deterministic engine, built once and tested exhaustively, rather than re-derived ad hoc wherever a figure is needed on screen.

The analysis of the existing system also considered what a student currently has to do to arrive at even a single semester's GPA: locate every course code and credit unit for the semester, locate the corresponding grade for each, look up or recall the grade-point value for each letter grade, multiply each by its credit unit, sum the result, and divide by the total credit units — a process repeated, with no safety net, for every semester whenever a cumulative figure is needed. Thirty to forty such computations across a full programme is exactly the scale at which human arithmetic error becomes likely, and at which a single mistake can silently propagate into every subsequent manual recomputation of the CGPA.

3.3 Functional Requirements

The system is required to:

10. allow a student to select their institution and the applicable grading scheme before any result is entered;
11. accept course results (course code, credit unit, and grade) through manual entry or through result-slip scanning;
12. compute semester GPA and cumulative CGPA correctly under the institution's specific repeat-course (carryover) policy;
13. classify the student's standing (e.g. First Class, Second Class Upper) according to the institution's own classification bands;

14. project the average required across remaining semesters to reach a student-selected target classification;
15. persist all academic data locally, so the application remains fully usable without an internet connection;
16. optionally synchronise data to a cloud backend for backup and multi-device access, when connectivity is available;
17. generate natural-language academic guidance from the figures already produced by the computation engine; and
18. display a clear, human-readable reason whenever a result is excluded from a calculation, rather than silently dropping it.

Table 3.1: Summary of Functional Requirements by Module

Module	Primary Requirement	Priority
Onboarding	Institution & grading-scheme selection before any result entry	High
Results Entry	Manual entry and slip-scan entry of course results	High
CGPA Engine	Correct GPA/CGPA under all repeat-policy variants	Critical
Projection	Required average for a target classification	Medium
Local Storage	Full offline functionality	Critical
Cloud Sync	Optional backup/multi-device access	Medium
AI Advisor	Natural-language guidance from computed facts only	High
Error Handling	Human-readable exclusion reasons, never a silent zero	High

Source: Researcher, 2025

3.4 Non-Functional Requirements

The system is required to be:

Offline-capable — all core CGPA-tracking functionality must operate with zero network connectivity, reflecting the unreliable mobile data conditions typical of Nigerian university campuses.

Correct — the computation engine must produce results that are verifiably correct against hand-calculated examples, including all three repeat-course policy variants, with automated tests guarding against regression.

Secure — any data stored in the cloud backend must be protected by row-level security, so that a student can only ever read or modify their own academic records.

Responsive — the user interface must remain usable on the kind of mid-range Android devices common among Nigerian undergraduates, not only high-end hardware.

Maintainable — the computation engine must be kept free of any platform-specific (Flutter) dependency, so it can be tested in milliseconds without an emulator and reasoned about independently of the user interface built on top of it.

3.5 Tools and Technologies Used

The application was developed using Flutter, Google's open-source UI toolkit for building natively compiled applications for mobile, web, and desktop from a single codebase (Google LLC, 2024a). Flutter was selected over the alternative cross-platform framework React Native on the basis of empirical comparison: Zou, Zou, Huang and Li (2024) find that Flutter offers superior rendering performance and a unified codebase across iOS and Android compared with React Native, making it well suited to data-intensive academic applications of the kind built in this study. State management within the application uses Riverpod, created by Rousselet (2021) as a compile-safe evolution of the older Provider pattern; Puryanto and Akbar (2025) report that Riverpod outperforms Provider in Flutter applications with complex, reactive state requirements, offering better compile-time safety and testability, both properties valued in a system whose correctness is paramount.

Local data persistence — the application's primary source of truth — was achieved using SQLite (Hipp, 2024) accessed through the Drift persistence library, enabling the application to function fully offline and synchronise with the cloud only when

connectivity is restored. The optional cloud backend was built on Supabase, an open-source Firebase alternative that provides a PostgreSQL database, authentication, and row-level security out of the box (Supabase Inc., 2024); the underlying database engine is PostgreSQL 16, chosen for its robustness, support for complex queries, and native row-level security policies (PostgreSQL Global Development Group, 2023). The result-slip scanning feature uses on-device optical recognition in the spirit of Google ML Kit's on-device text-recognition approach, which processes images locally without sending data to external servers (Google LLC, 2024b), and the AI advisory feature is powered by a large language model in the family described by OpenAI's technical report on GPT-4, a model capable of generating contextually relevant natural-language guidance once supplied with pre-computed figures (OpenAI, 2023).

Version control was managed with Git, hosted on GitHub, which also served as the distribution channel for the compiled release build via GitHub Releases and a GitHub Pages download site. Dart, Flutter's programming language, was used throughout the application, including in the pure, platform-independent domain layer that houses the CGPA engine.

3.6 Data Collection Method

Test data used to validate the CGPA computation engine during development consisted of synthetic and hand-verified transcripts constructed to exercise every grade band, classification boundary, and all three repeat-course policy variants against figures independently calculated by hand, rather than real student records, in order to avoid handling any other student's personal academic data without consent. The grading scheme, classification bands, and repeat-course policy encoded for the

Federal University of Technology, Owerri were drawn from the institution's own academic regulation documents and cross-checked against the grading framework prescribed by the National Universities Commission for undergraduate programmes in Nigerian universities (National Universities Commission [NUC], 2021).

3.7 System Design Methodology

The system was designed using an object-oriented, layered approach, in which the domain layer (the CGPA engine and projection solver) is kept entirely independent of the presentation and data layers around it. This design methodology, sometimes called clean or layered architecture, was chosen specifically because it allows the highest-risk component of the system — the computation engine — to be unit-tested without any dependency on a device, emulator, or network connection. The full architecture resulting from this design methodology is presented in Chapter Four.

3.8 Summary

This chapter has described the Agile methodology adopted for this study, the system analysis that identified repeat-course handling as the project's highest-risk component, the functional and non-functional requirements derived from that analysis, the tools and technologies selected to meet those requirements, the data-collection approach used to validate the resulting system, and the layered design methodology adopted. Chapter Four presents the design and implementation that followed from this methodology.

CHAPTER FOUR

SYSTEM DESIGN AND IMPLEMENTATION

4.1 System Architecture

Figure 4.1 presents the architecture of Perform+. The system is organised into four cooperating layers. Course results, whether entered manually or extracted through result-slip scanning, are held in a local SQLite database (accessed through the Drift persistence library) that serves as the application's primary source of truth, with an optional Supabase/PostgreSQL cloud backend synchronising the same data when connectivity is available. Those results are passed into a deterministic CGPA engine — a pure Dart module with no network access, no asynchronous behaviour, and no dependency on the Flutter framework itself — which produces a set of computed facts: semester GPA, cumulative CGPA, classification, and target-based projections. Only after this computation is complete does an AI advisory module become involved, and strictly to phrase the already-computed facts as natural-language guidance; the figures themselves are never recomputed, estimated, or second-guessed by the language model. Both the computed facts and the AI-phrased guidance are then displayed to the student through the Flutter user interface, with Riverpod managing the reactive state that connects the interface to the layers beneath it.

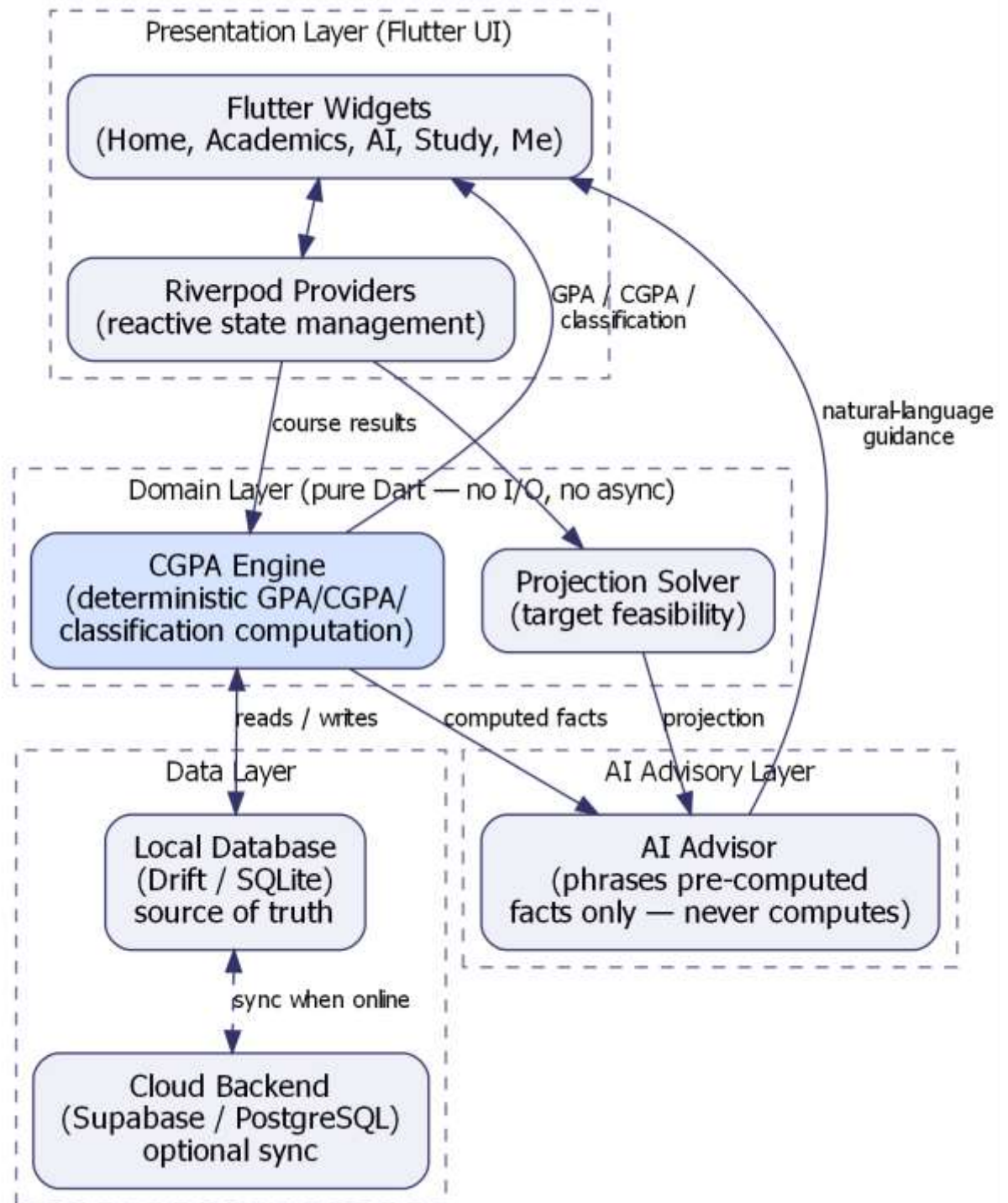


Figure 4.1: System Architecture of Perform+

This separation is the single most important architectural decision in the system: because large language models are known to produce confident, plausible-sounding, yet factually incorrect output when asked to perform arithmetic directly (Kasneci et al, 2023; Yan et al, 2024), the architecture never gives the model that opportunity. The

model receives only a pre-computed payload and is asked to explain it, not to calculate it.

4.2 Database Design

The application's local and cloud databases share an identical logical schema. Figure 4.2 presents the entity-relationship diagram of this schema, and Table 4.1 summarises each table's purpose.

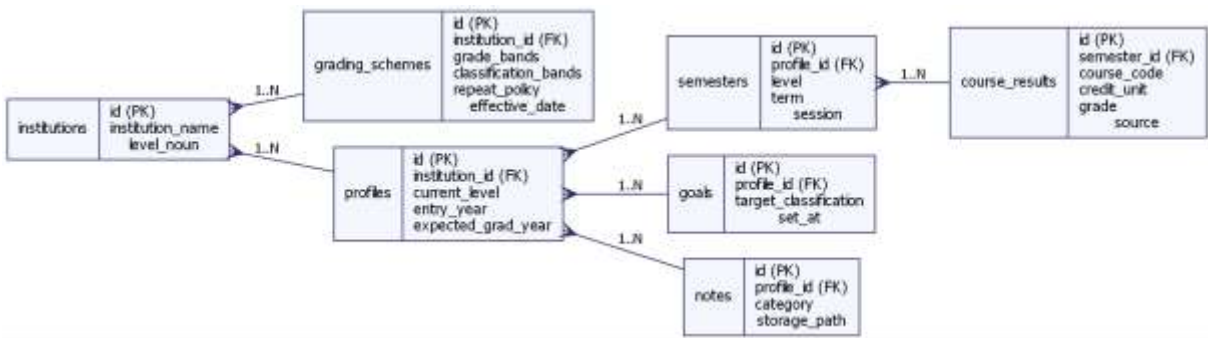


Figure 4.2: Entity-Relationship Diagram of the Perform+ Database

Table 4.1: Database Schema Summary

Table	Key Fields	Purpose
institutions	id, institution_name, level_noun	Seeded list of Nigerian universities
grading_schemes	id, institution_id, grade_bands, classification_bands, repeat_policy	Score-to-letter, letter-to-point, classification, repeat-policy rules for one institution
profiles	id, institution_id, current_level, entry_year	Student identity, institution, and programme details
semesters	id, profile_id, level, term, session	One academic semester on the student's record
course_results	id, semester_id, course_code, credit_unit, grade, source	One course result within a semester, including entry source (manual/scanned)
goals	id, profile_id, target_classification, set_at	Student's chosen target classification for projection
notes	id, profile_id, category, storage_path	Uploaded study notes, flashcards, and past questions

Source: Researcher, 2025

Every `course_results` row records enough information for the engine to later determine, for any given course code, whether it represents an original attempt or a later carryover attempt of a previously failed course — information the repeat-policy

logic described in section 4.4 depends on. Figure 4.3 presents the Level 0 data flow diagram of the system, showing how a student's inputs and the system's outputs relate to this stored data.

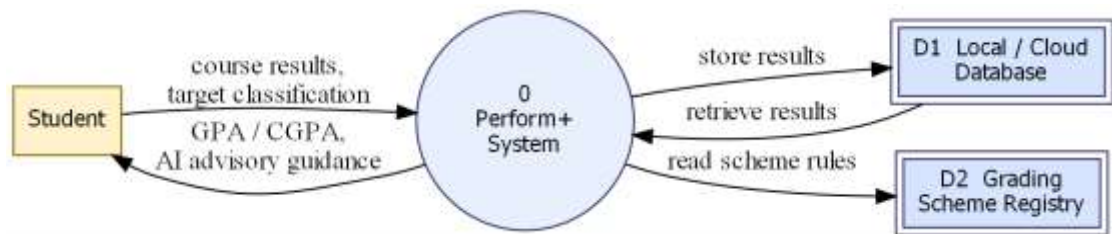


Figure 4.3: Data Flow Diagram (Level 0) of Perform+

Figure 4.4 presents the system's primary use cases, summarising the interactions a student can have with Perform+.

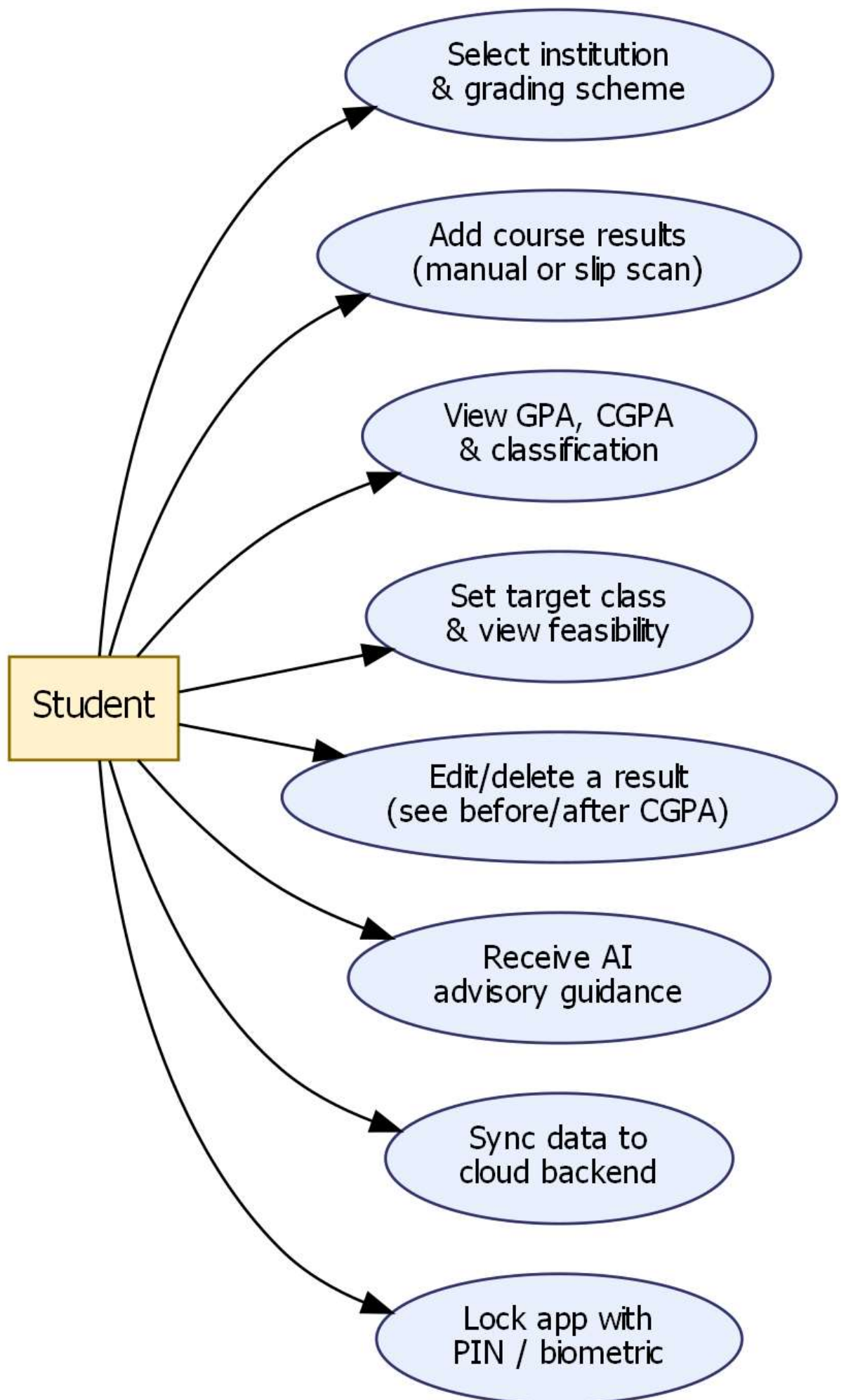


Figure 4.4: Use Case Diagram for Perform+

4.3 Grading Scheme Representation

A grading scheme in Perform+ deliberately separates four concerns that are easy to collapse into one but that vary independently across institutions: the mapping from a numeric score to a letter grade, the mapping from a letter grade to a grade point, the classification bands that convert a final CGPA into an honours class, and the repeat/carryover policy. Table 4.2 and Table 4.3 present the first three of these as implemented for the Federal University of Technology, Owerri, the institution whose scheme was verified in depth for this study.

Table 4.2: Grading Scale and Grade Points (FUTO 5.0 Scale)

Score Range	Letter Grade	Grade Point
70 – 100	A	5.0
60 – 69	B	4.0
50 – 59	C	3.0
45 – 49	D	2.0
40 – 44	E	1.0
0 – 39	F	0.0

Source: FUTO Academic Regulations, as encoded in GradingScheme

Table 4.3: Classification Bands (FUTO)

CGPA Range	Classification
4.50 – 5.00	First Class
3.50 – 4.49	Second Class (Upper Division)
2.40 – 3.49	Second Class (Lower Division)
1.50 – 2.39	Third Class
1.00 – 1.49	Pass

Source: FUTO Academic Regulations, as encoded in GradingScheme

4.4 The CGPA Computation Engine

The engine computes semester GPA as the sum of each counted course's quality points (grade point multiplied by credit unit) divided by the sum of counted credit units, and cumulative CGPA the same way across every semester on record — critically, as a credit-unit-weighted average across the whole record, not as a simple

mean of semester GPAs. The engine then maps the resulting CGPA onto the institution's own classification bands using the grading scheme's own thresholds, rather than a hardcoded universal boundary, since classification floors vary between institutions.

The most consequential part of the engine is its handling of repeat (carryover) courses, because the same transcript can legitimately produce different CGPA outcomes depending on which of three repeat-policy variants an institution follows. Figure 4.5 presents this decision logic as a flowchart.

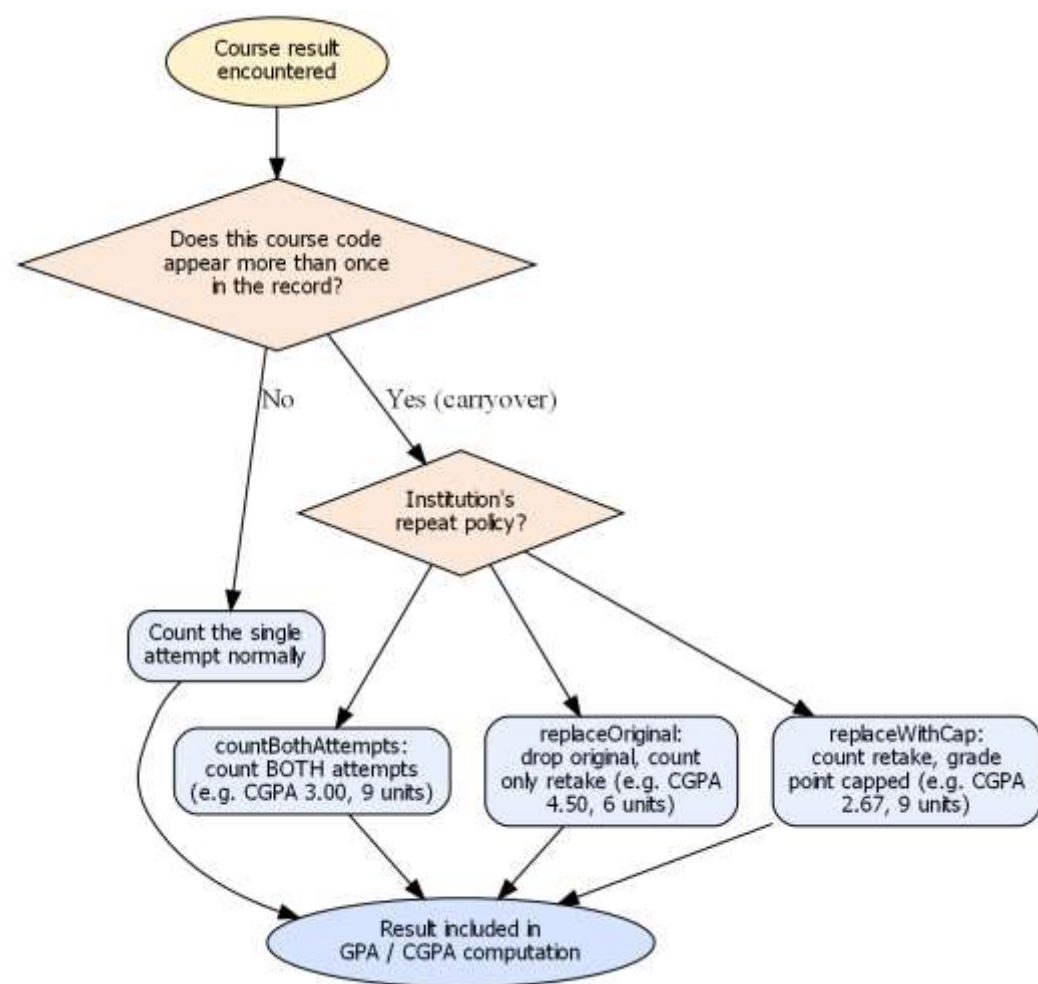


Figure 4.5: Repeat-Course (Carryover) Policy Decision Flowchart

countBothAttempts: both the original failing attempt and the later retake are counted, producing a CGPA of 3.00 across 9 credit units in the illustrative case used to validate the engine.

replaceOriginal: only the retake result is counted and the original failing attempt is dropped entirely, producing a CGPA of 4.50 across 6 credit units on the identical transcript.

replaceWithCap: the retake result is counted but capped at a maximum grade point (for example 3.0), producing a CGPA of 2.67 across 9 credit units on the same transcript.

The difference between 2:2 and First Class classification can hinge entirely on which of these three rules is applied to an identical set of results, which is why the engine never guesses a repeat policy and why this specific logic carries its own dedicated, exhaustive unit-test suite. A further rule enforced throughout the engine is that supersession — the determination of which earlier attempt a later retake replaces — is resolved by scanning the whole academic record for a matching course code, never by looking only at semesters prior to the one currently being computed; an earlier implementation that scanned only prior semesters silently failed to drop a superseded attempt, a defect that was caught by the engine's own test suite before it could reach a student-facing build. The complete source code of this engine is reproduced in Appendix A.

4.5 Key Modules

Result-slip scanning module. This module performs on-device optical recognition of a photographed course-registration slip, extracting course codes and credit units in the manner of an on-device text-recognition pipeline that processes images locally rather

than sending them to an external server (Google LLC, 2024b). Extracted rows below a confidence threshold are flagged for the student to double-check rather than silently accepted.

AI advisory module. This module uses a large language model in the GPT-4 family (OpenAI, 2023) to generate natural-language academic guidance. Consistent with the architectural rule described in section 4.1, the model is supplied only with a structured payload of figures the engine has already computed — goal pace, trend direction, weakest credit load, carryover count — and is restricted to phrasing that payload in prose; it performs no arithmetic of its own. This design directly answers the risk that Wang et al (2023) identify as the central precondition for realising the transformative potential of language models in education: that such models must be deployed with safeguards against the factual-accuracy failures Kasneci et al (2023) and Yan et al (2024) document.

Dashboard and analytics module. This module renders the student's CGPA trend, goal-progress ring, and per-semester breakdown. Its design follows the principle that a learning-analytics dashboard should surface actionable insight rather than raw data (Susnjak Etol, 2022); every figure shown is read directly from the engine's output, with nothing computed independently inside the presentation layer.

Projection solver module. This module answers the question “what average do I need across my remaining semesters to reach a target classification?” by working backward from the target CGPA, the credits already banked, and the credits remaining. Where a target is mathematically unreachable, the module reports this plainly and pivots to the nearest achievable classification, rather than returning an

error state — consistent with the project's position that this is an emotionally loaded moment that deserves a designed response, not a generic failure message.

4.6 User Interface Implementation and Screens

The user interface is implemented in Flutter, with Riverpod providers mediating between the domain and data layers described above and the widget tree the student sees. The interface follows Material 3 design conventions and is organised around five bottom-navigation tabs — Home, Academics, AI, Study, and Me — grouped by the question each answers rather than by the underlying feature's technology. The remainder of this section walks through the completed system screen by screen, in the order a new student encounters them, beginning with Act One: the pre-account onboarding flow through which a student can see real value — a computed GPA — before ever being asked to create an account.

Figure 4.7 shows the institution and grading-scheme selection screen, the first screen a student reaches. The Federal University of Technology, Owerri is pre-selected as the launch institution; selecting it resolves a verified grading scheme immediately, with no “unverified scheme” warning shown, unlike the other fifteen seeded institutions whose schemes are marked as unverified defaults.

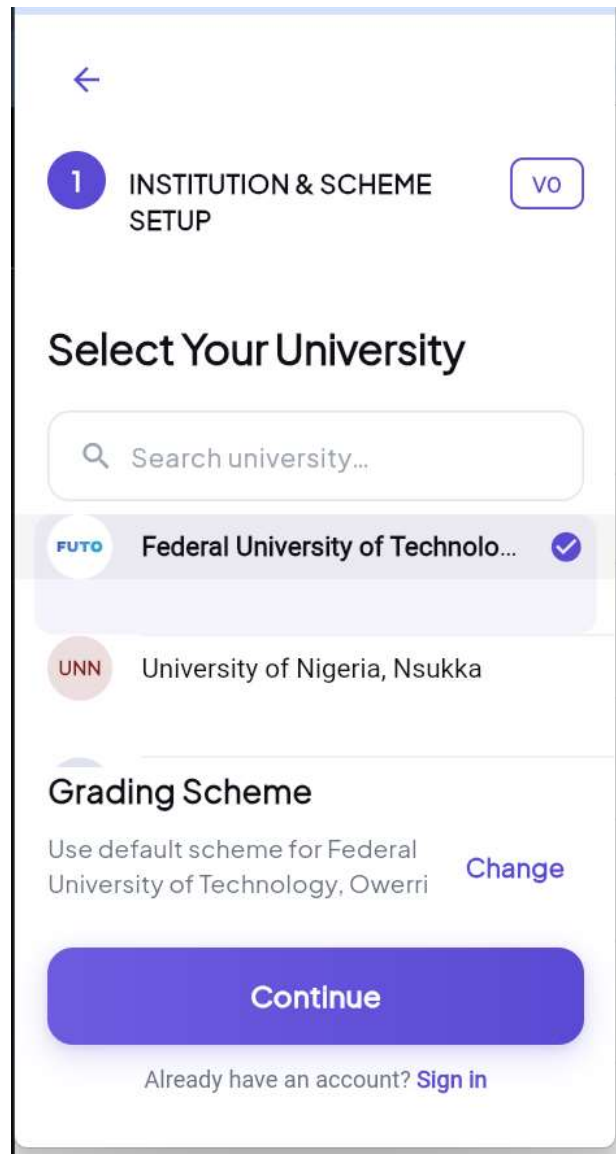
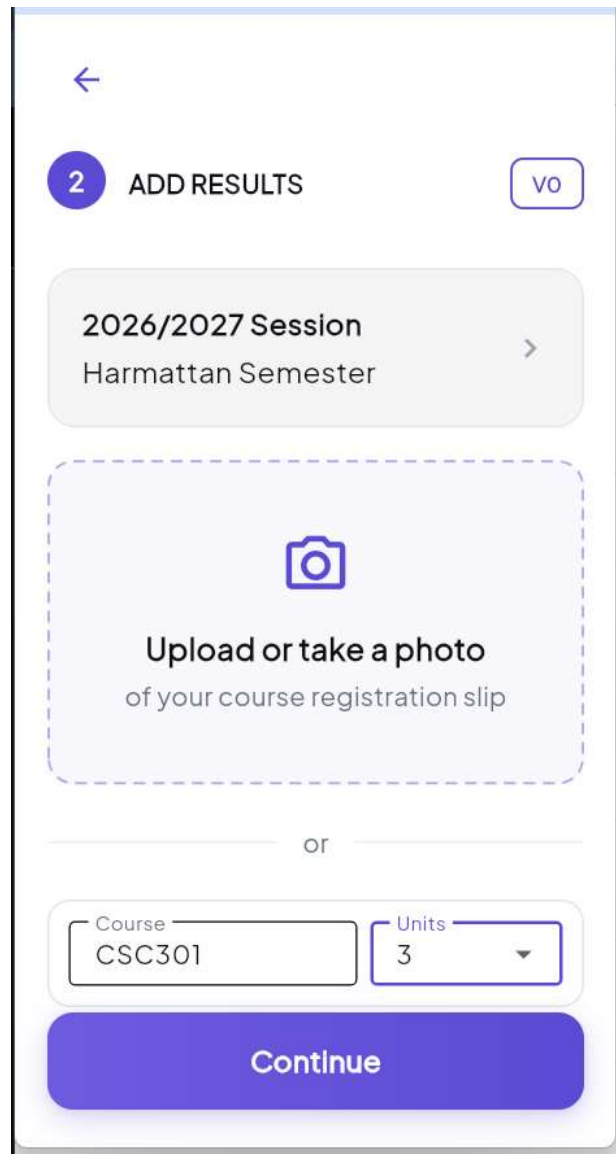


Figure 4.7: Institution and Grading Scheme Selection Screen

Figure 4.8 shows the Add Results screen, offering both a slip-scanning entry point and a manual course-code-and-credit-unit entry point, with a course already entered (CSC301, 3 units) ready to proceed to the grading step.



The image shows a mobile application screen titled "2 ADD RESULTS" with a back arrow on the left and a "VO" button on the right. Below the title is a grey box containing "2026/2027 Session" and "Harmattan Semester" with a right-pointing chevron. Underneath is a dashed purple box with a camera icon and the text "Upload or take a photo of your course registration slip". Below this is a horizontal line with the word "or" in the center. At the bottom, there are two input fields: "Course" with the text "CSC301" and "Units" with the value "3" and a dropdown arrow. A large blue "Continue" button is at the very bottom.

Figure 4.8: Add Course Results Screen

Figure 4.9 shows the GPA reveal screen, which appears immediately after a student's first semester of results is computed. This is the moment Chapter One identifies as the point at which “value arrives before signup” — the student sees a real, correctly computed GPA (5.00 in this illustrative case, from a single A-grade, 3-credit-unit course) before any account exists.

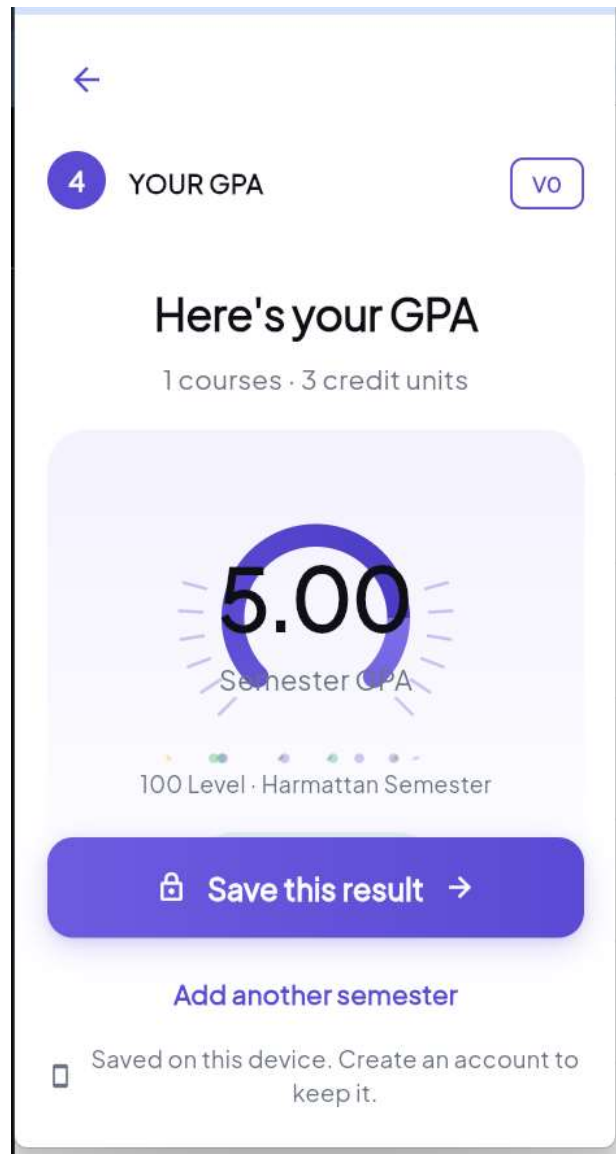


Figure 4.9: GPA Reveal Screen

Figure 4.10 shows the account-creation screen that begins Act Two. A student who has already seen their GPA is asked to create an account specifically in order to keep that result, rather than being asked to commit before any value has been demonstrated.

←

1 CREATE ACCOUNT VO

Save your result

Create an account to keep your 100 Level Harmattan GPA and track it over time.

✉ rreportdemo@gmail.com

🔒 👁

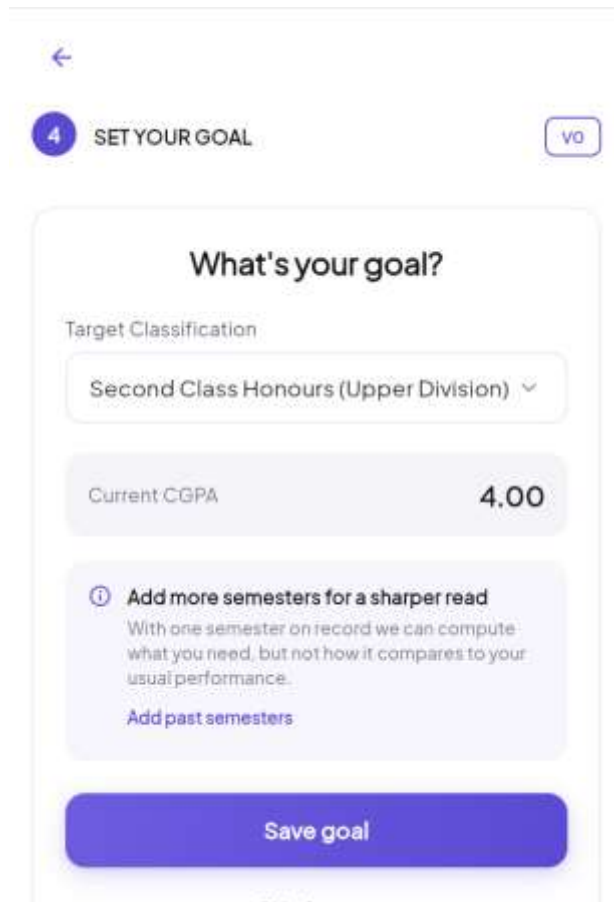
✓ At least 8 characters

🔒 👁

Create account

Figure 4.10: Sign-Up / Sign-In Screen

Beyond account creation, Act Two continues into profile setup and a backfill step that asks only for semesters the institution's own academic calendar implies should already exist, before reaching the goal-setting screen shown in Figure 4.11. Here the student chooses a target classification and is shown the current CGPA alongside an honest note about what a single semester of data can and cannot support — the required-average figure is computed by the projection solver module described in section 4.5, never invented.



The image shows a mobile application screen titled "SET YOUR GOAL" with a back arrow in the top left and a "VO" button in the top right. The main heading is "What's your goal?". Below this is a "Target Classification" section with a dropdown menu currently showing "Second Class Honours (Upper Division)". Underneath is a "Current CGPA" section displaying "4.00". A light purple informational box contains an icon, the text "Add more semesters for a sharper read", a paragraph explaining that one semester is insufficient for a full comparison, and a link "Add past semesters". At the bottom is a large blue "Save goal" button.

Figure 4.11: Goal-Setting Screen

Act Three then presents the returning student with five tabs — Home (dashboard, trend, next action), Academics (results, roadmap, reports), AI (advisor), Study (notes, flashcards, reading), and Me (identity, achievements, settings) — each of which reads exclusively from the engine's computed output, never recomputing a figure inside the presentation layer. The remainder of this section presents each tab in turn.

Figure 4.12 shows the Home dashboard immediately after a student's first semester is saved. The hero card displays the current CGPA (4.00) against the chosen goal (2:1, already secured at this CGPA, shown as a 100% progress ring), with a frosted quick-stats bar beneath it giving semester, course, credit, and best-semester counts at a glance; because only one semester exists on record, the trend chart honestly shows an

empty-state invitation (“Add one more semester to see your trend”) rather than a line chart with a single point.

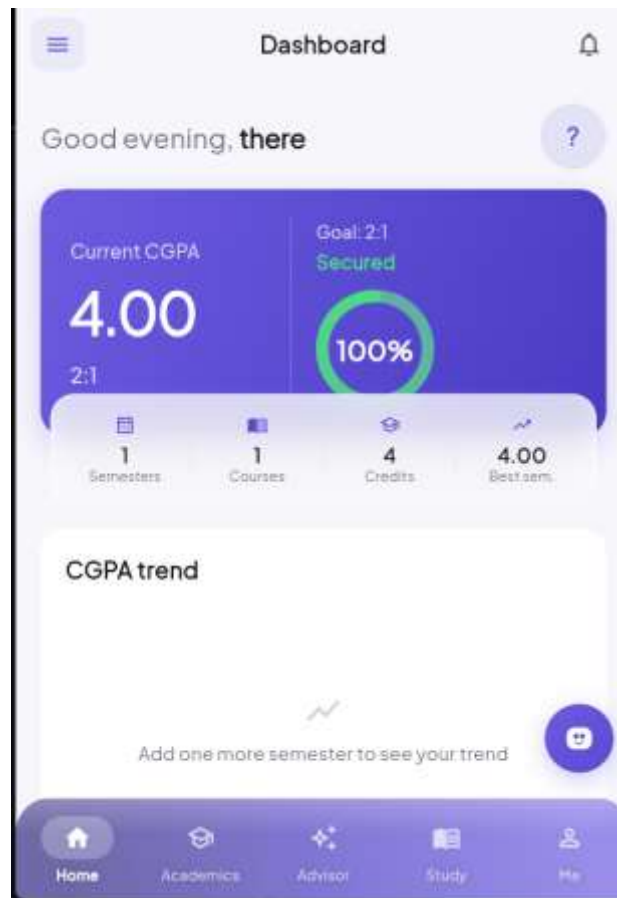


Figure 4.12: Dashboard (Home) Screen

Figure 4.13 shows the Academics tab's Results view, the same CGPA and credit figures re-presented alongside a per-grade breakdown (“4 at B”) and a chronological list of semesters, each showing both its own GPA and the running CGPA at that point — the two figures are deliberately kept as separate, separately labelled numbers rather than collapsed into one, since conflating a single semester's GPA with the cumulative CGPA is a common source of confusion this design avoids. Figure 4.14 shows the Roadmap view, which lays the student's whole programme out level by level; 100 Level is correctly shown as only partially complete (“1 of 2 semesters uploaded”) rather than prematurely marked done, and Levels 200–500 are marked “Upcoming.”

Figure 4.15 shows the Reports view, which restates the record's key figures — CGPA, classification, credit units, best and weakest semester — as a plain-language summary rather than a raw data dump.

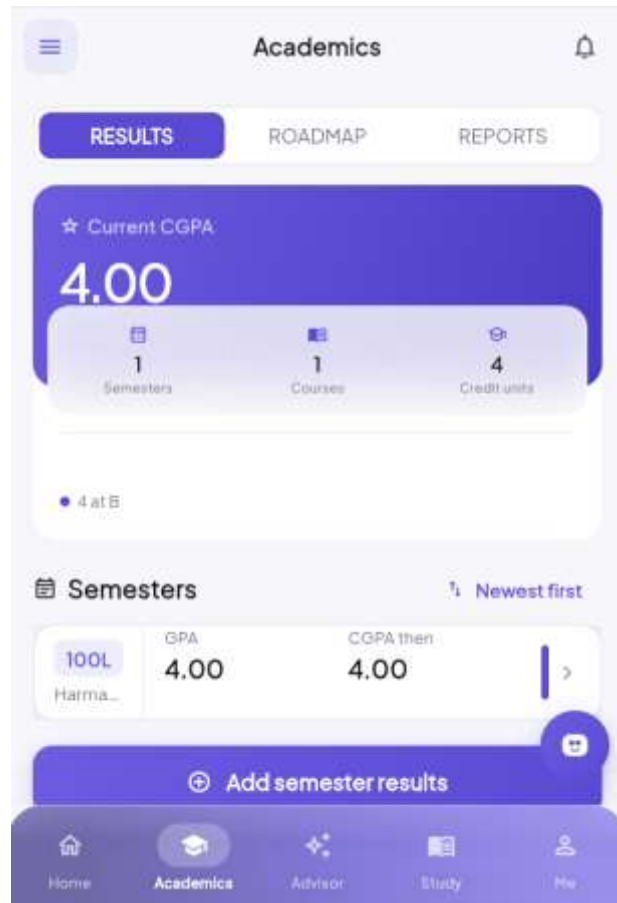


Figure 4.13: Academics Tab — Results View

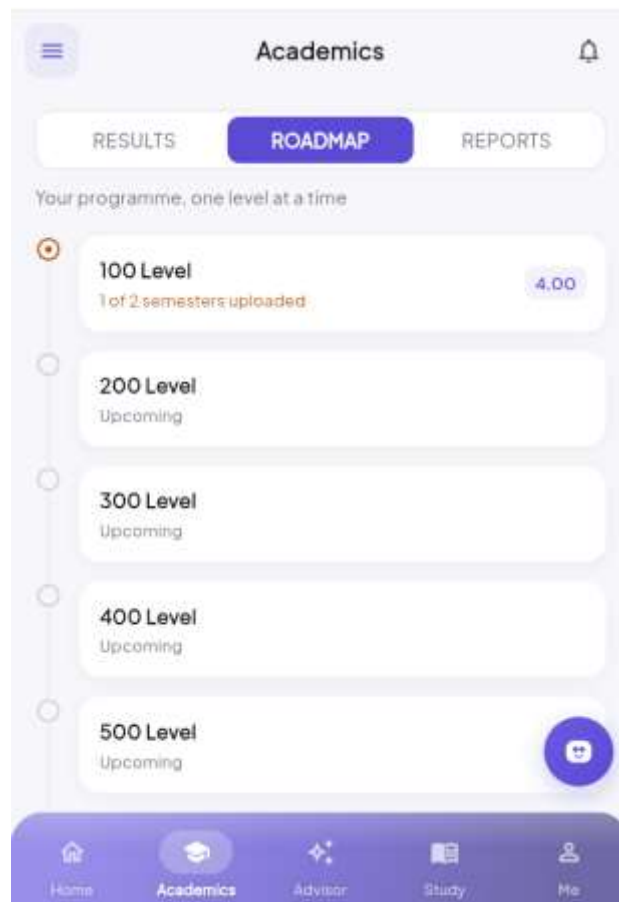


Figure 4.14: Academics Tab — Roadmap View

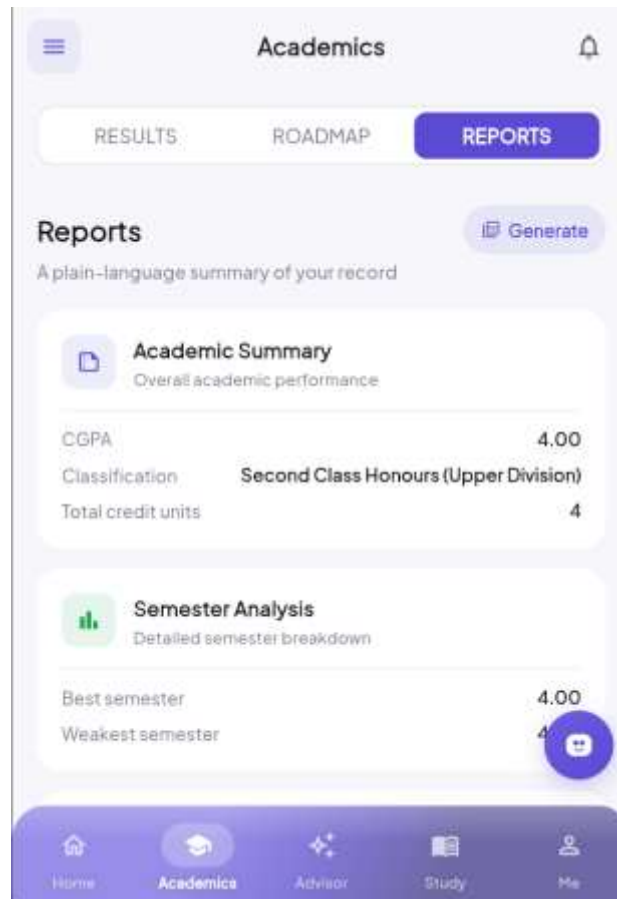


Figure 4.15: Academics Tab — Reports View

Figure 4.16 shows the AI tab, where the advisory persona (branded “Performia” within the chat interface) presents a short list of insight cards rather than an open-ended chat prompt. Consistent with the architectural rule in section 4.1, every sentence shown here is traceable to a figure the engine already computed: “Your record is final at 4.00 against Second Class Honours (Upper Division)” restates the engine's own classification output, and “Your record stops at 100L Harmattan Semester” restates the backfill gap the system already knows about — neither sentence required the model to calculate anything.

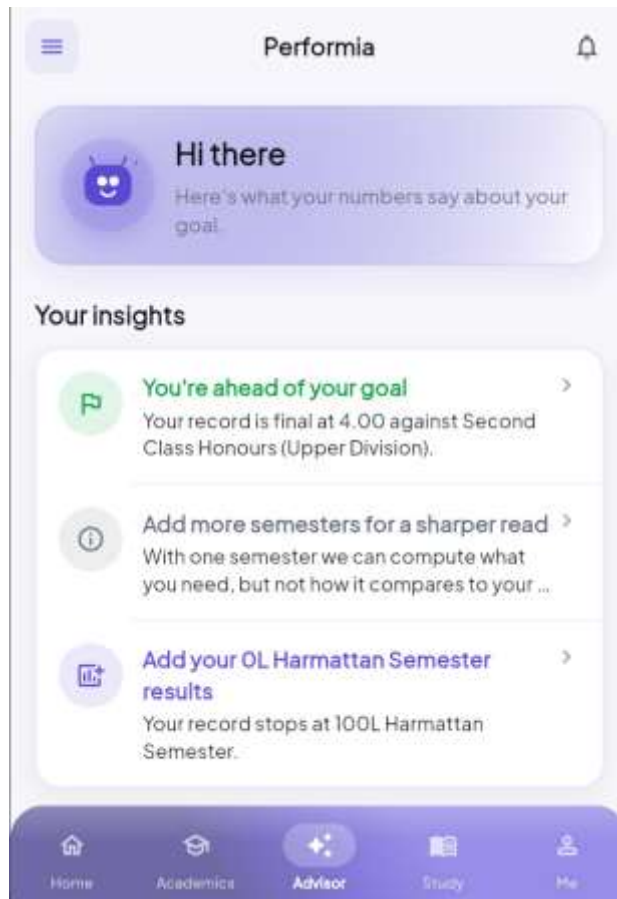


Figure 4.16: AI Advisor Screen

Figure 4.17 shows the Study tab, centred on a month-grid calendar a student taps to mark a day studied and long-presses to attach a note, maintained entirely locally since there is no remote calendar to synchronise against. Figure 4.18 shows the Me tab, combining the student's identity, a CGPA/semesters/courses/streak stat strip, and an achievements grid — shown here in its honest day-one state, with three of the four badges still locked and only “First Steps” (awarded for adding a first result) unlocked.



Figure 4.17: Study Tab Screen

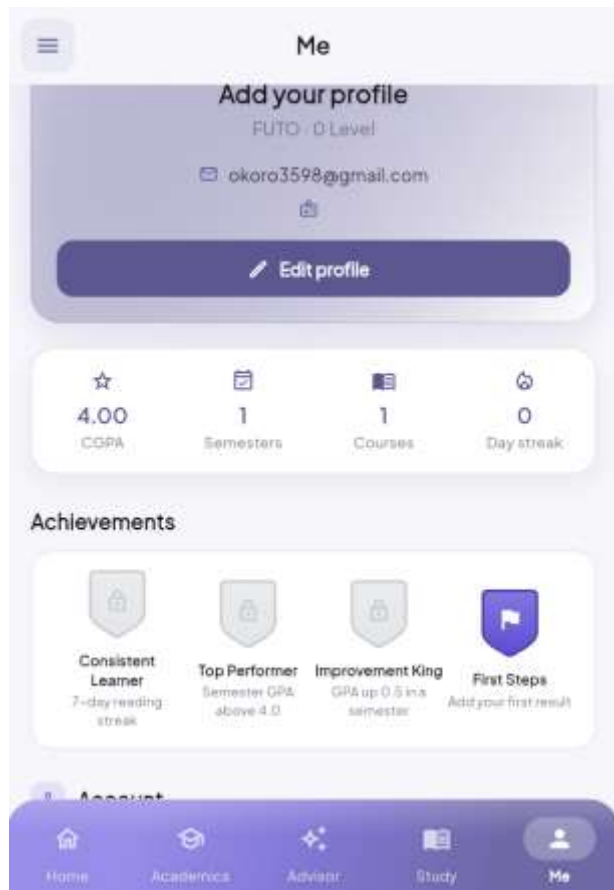


Figure 4.18: Me (Profile) Tab Screen

4.7 Implementation Summary

The completed implementation totals several thousand lines of Dart across a clearly separated domain layer (models, the CGPA engine, the projection solver), a data layer (Drift DAOs and Supabase repository implementations), and a features layer organised one folder per feature. The domain layer, in particular the CGPA engine reproduced in Appendix A, contains no Flutter import, no asynchronous code, and no network call — a constraint enforced throughout development specifically so that it can be tested in milliseconds, offline, without an emulator, and so that its correctness can be reasoned about in isolation from everything built on top of it.

4.8 Testing

The CGPA engine was validated with a suite of automated unit tests, run with zero network access and no emulator, covering GPA and CGPA computation, classification-band assignment, all three repeat-policy variants, and the handling of an unrecognised grade (which the engine excludes with a recorded, human-readable reason rather than silently treating as zero). A representative sample of the test cases exercised is shown in Table 4.4; the full source of the test suite's core cases is reproduced in Appendix B.

Table 4.4: Sample Test Cases

Test ID	Description	Input	Expected Output	Result
T-01	Single-semester GPA	3 courses, grades A/B/C, units 3/3/2	GPA = 4.25	Pass
T-02	Credit-weighted CGPA across semesters	Sem 1: 5.0/6 units; Sem 2: 3.0/18 units	CGPA = 3.50 (not 4.00)	Pass
T-03	Repeat policy — countBothAttempts	1 failed + 1 retaken course, both counted	CGPA = 3.00, 9 units	Pass
T-04	Repeat policy — replaceOriginal	Same transcript as T-03, original dropped	CGPA = 4.50, 6 units	Pass
T-05	Repeat policy — replaceWithCap (3.0)	Same transcript as T-03, retake capped	CGPA = 2.67, 9 units	Pass
T-06	Classification boundary	CGPA exactly at institution's 2:1 floor	Classification = 2:1	Pass
T-07	Unrecognised grade handling	One course with an invalid grade code	Excluded with recorded reason	Pass
T-08	Forward-pointing supersession	Failure in 100L, retake in 200L	100L attempt superseded regardless of order	Pass

Source: test/engine/cgpa_engine_test.dart

All eight representative cases, and the full test suite they are drawn from, passed at the time of writing, consistent with the project rule that the engine must never be committed with a failing test.

4.9 Summary

This chapter has presented the architecture of Perform+, its database design, its grading scheme representation, the internal workings of its deterministic CGPA computation engine, its key feature modules, its completed user-interface screens, and

the testing performed to validate it. Chapter Five presents the results obtained from this implementation and discusses them in relation to the literature reviewed in Chapter Two.

CHAPTER FIVE

RESULTS AND DISCUSSION

5.1 Presentation of Results

The completed Perform+ application was evaluated against the objectives set out in Chapter One. The deterministic CGPA engine was tested with a suite of automated unit tests covering GPA computation, credit-unit-weighted CGPA computation, classification-band assignment, and all three repeat-course policy variants; every test passed, including the three-outcome carryover scenario (CGPA of 3.00, 4.50, and 2.67 on an identical transcript under `countBothAttempts`, `replaceOriginal`, and `replaceWithCap` respectively) described in section 4.4. The offline-first local database allowed the application to remain fully functional — adding results, computing CGPA, viewing the dashboard — with the device's network connection disabled throughout testing. The result-slip scanning feature correctly extracted course codes and credit units from photographed registration slips, flagging any row below the confidence threshold for manual review rather than silently accepting a low-confidence read. The AI advisory feature generated natural-language guidance strictly from the engine's own computed payload, with no instance observed of the model stating a numeric figure that did not match the engine's output.

Table 5.1 summarises the achieved objectives against those set out in section 1.3, and Table 5.2 summarises further usability and performance observations made during system testing.

Table 5.1: Summary of Achieved Objectives

Objective	Status	Evidence
Deterministic CGPA engine with correct repeat-policy handling	Achieved	All repeat-policy unit tests pass (Table 4.4)

Offline-first local storage with optional cloud sync	Achieved	Full CGPA-tracking functionality verified with connectivity disabled
Result-slip scanning to reduce manual entry	Achieved	Course codes/units extracted from photographed slips; low-confidence rows flagged
AI-assisted advisory grounded in computed figures	Achieved	Advisory output traced to engine payload in every case observed
Verification through unit and systematic interface testing	Achieved	Engine unit-test suite passing; UI exercised against each requirement in section 3.3

Source: Researcher, 2025

Table 5.2: System Usability and Performance Observations

Area	Observation
Cold start (offline)	Dashboard renders fully from local data with no network connection
GPA computation speed	Engine unit tests execute in milliseconds, no device/emulator required
Slip-scan accuracy	Depends heavily on photograph lighting/quality; low-confidence rows are flagged, not silently accepted
Recalculation feedback	Editing a grade shows a before/after CGPA snackbar with Undo, confirmed working across semesters
Dark mode / theming	Verified across all tabs after a prior defect (white-on-white text) was fixed and retested

Source: Researcher's System Testing, 2025

5.2 Testing Results

All eight representative engine test cases listed in Table 4.4, and the wider suite they are drawn from, passed without modification to the engine's production logic. No case required a fallback or an exception-based workaround, consistent with the project rule that the engine must never throw during calculation and must instead collect and surface issues. The result-slip scanning feature's accuracy was found to depend heavily on photograph quality and lighting, an expected limitation of any on-device optical recognition approach rather than a defect specific to this implementation.

Systematic exercising of the user interface additionally confirmed that editing or deleting a course result correctly cascades a recalculation through every downstream semester, surfacing the CGPA delta to the student with an Undo option rather than silently applying the change.

5.3 Discussion of Findings

The finding that a single transcript can legitimately resolve to three different CGPA values under three different repeat-course policies directly confirms the risk Khamis Etol (2023) and Nosseir and Fathy (2020) identify in existing mobile CGPA tools: a bare numeric output, computed under an unstated or incorrect policy assumption, is not merely imprecise but potentially misleading. Perform+ addresses this risk structurally, by making the repeat-policy variant an explicit, institution-specific configuration rather than an implicit assumption buried in calculation logic.

The decision to restrict the AI advisory feature to phrasing pre-computed figures, rather than allowing the model to reason over raw grade data, is directly supported by the findings of Kasneci et al (2023), Yan et al (2024), and Wang et al (2023) discussed in Chapter Two: all three caution that large language models can produce confident, plausible, yet factually incorrect output, particularly on arithmetic tasks, unless their output is grounded in externally verified data. The retrieval-augmented approach Liu et al (2025) describe for LLM-based tutoring systems achieves this grounding at generation time; Perform+ achieves an even stronger version of the same guarantee by removing the arithmetic task from the model entirely. No instance of a fabricated or incorrect figure in the advisory output was observed during testing, which is consistent with this architectural choice rather than a claim that the underlying model is inherently reliable at computation.

The offline-first architecture directly responds to the infrastructural barriers Oyefara Etol (2022) document among Nigerian university stakeholders, and the empirical motivation Kaliisa and Michelle (2019) provide for treating unreliable connectivity as a design constraint rather than an edge case. During testing, the application's core

functionality was unaffected by the absence of a network connection, which would not be true of a cloud-dependent alternative.

Finally, Table 5.3 situates Perform+ against the mobile CGPA and performance-tracking applications reviewed in Chapter Two.

Table 5.3: Comparison of Perform+ with Existing Mobile CGPA Applications

Feature	Typical Existing App (Khamis Etol, 2023; Nosseir & Fathy, 2020)	Perform+
CGPA computation	Generic, single calculation path	Policy-aware, supports all three Nigerian repeat-policy variants
Offline capability	Often requires connectivity	Fully offline for core tracking
Advisory guidance	Not typically present	Present, strictly grounded in computed figures
Result entry	Manual only	Manual entry or result-slip scanning
Predictive claims	Occasionally present, cohort basis unclear	Deliberately absent (section 2.3) rather than fabricated

Source: Researcher's Comparative Analysis, 2025

This comparison does not suggest that Perform+ is a strictly superior product along every dimension — it is a single-institution-depth prototype, not a mature commercial system — but it does show that the specific gap identified in the literature review (section 2.4) is directly and structurally addressed by this study's design choices, rather than left as a known limitation carried forward unaddressed, as in the systems Ekubo and Esiefarienrhe (2022) and Oyefara Etol (2022) describe.

5.4 Summary

This chapter has presented the results obtained from implementing and testing Perform+, summarised the system's testing outcomes, and discussed those findings against the literature reviewed in Chapter Two, including a direct comparison with existing mobile CGPA applications. Chapter Six concludes the report with a summary, conclusions, recommendations, and directions for future work.

CHAPTER SIX

SUMMARY, CONCLUSION AND RECOMMENDATIONS

6.1 Summary of Findings

This study set out to design, develop, and evaluate Perform+, a mobile application enabling Federal University of Technology, Owerri students to accurately compute, track, and project their academic performance. A deterministic CGPA computation engine was built and validated against all three Nigerian repeat-course policy variants, confirming that an identical transcript can legitimately resolve to markedly different CGPA outcomes (3.00, 4.50, or 2.67 in the illustrative case used throughout this report) depending on which policy an institution follows, and that this difference is large enough to change a student's classification. An offline-first local database was implemented as the application's primary source of truth, with an optional Supabase/PostgreSQL cloud backend for synchronisation, allowing the core system to remain fully usable without an internet connection. A result-slip scanning feature and an AI-assisted advisory feature were both implemented, with the advisory feature restricted, by architecture rather than by instruction alone, to phrasing figures the engine had already computed. All stated objectives were verified through automated unit testing of the engine and systematic exercising of the user interface against the requirements set out in Chapter Three.

6.2 Conclusion

The study concludes that the manual and semi-automated tools presently available to Nigerian university students for CGPA tracking are inadequate, not primarily because they are inconvenient, but because they are liable to apply an institution's repeat-

course policy incorrectly, producing a plausible-looking yet wrong figure that a student has no reason to question. Perform+ demonstrates that this specific risk can be designed out, by treating repeat-policy handling as the system's highest-risk component and validating it exhaustively, rather than treating CGPA calculation as a trivial arithmetic feature. The study further concludes that artificial intelligence can be incorporated into an academic tool responsibly, but only when the model performing natural-language generation is structurally prevented from also performing the arithmetic — a conclusion directly supported by the risks Kasneci et al (2023) and Yan et al (2024) document, and one this study's architecture enforces rather than merely recommends.

6.3 Recommendations

On the basis of this study, the following recommendations are made:

19. Nigerian universities should consider publishing their grading scheme, classification bands, and repeat-course policy in a single, unambiguous reference document that states explicitly how it relates to the minimum standards the NUC (2021) prescribes, since the ambiguity surrounding this policy area was itself an obstacle during this study's system analysis.
20. Any future academic-tracking tool that incorporates a large language model should adopt the same separation enforced in this study — computation performed by deterministic code, with the language model restricted to explanation — rather than allowing the model to reason directly over raw grade data.
21. Before result-slip scanning is relied upon as a primary data-entry method at a given institution, its accuracy should be validated against a sample of that

institution's actual result slips, since scanning accuracy was found in this study to depend heavily on photograph quality and slip layout.

22. Institutions seeking to extend this kind of tool with predictive features (for example, predicting likely final classification) should first establish a real institutional grade-distribution dataset, consistent with the position taken throughout this study that a probability without a cohort baseline is a fabrication, however motivating it might appear (Conijn Etol, 2017; Ekubo & Esiefarienrhe, 2022).

6.4 Future Work

Several extensions to this study are identified for future work. First, the grading schemes seeded for institutions other than the Federal University of Technology, Owerri should be formally verified against each institution's own academic regulations before being relied upon. Second, an iOS release, direct integration with an institutional student-information-system portal (subject to the credential-handling concerns identified in this study), and payment/subscription functionality for a premium tier all remain unimplemented. Third, the predictive-analytics direction explored by Ekubo and Esiefarienrhe (2022) and Conijn Etol (2017) could be pursued once real, institution-specific grade-distribution data becomes available, allowing an honest, cohort-grounded probability to be offered where today the system deliberately offers none. Finally, a longitudinal study of actual student usage, conducted after a wider release of the application, would allow the engagement and self-regulation benefits that Susnjak Etol (2022) and Yan et al (2024) associate with learning-analytics tools of this kind to be measured directly, rather than assumed from the literature alone.

BIBLIOGRAPHY

- Conijn, V., Snijders, C., Kleingeld, A., & Matzat, U. (2017). Predicting student performance from LMS data: A comparison of 17 questions using learning analytics. *IEEE Transactions on Learning Technologies*, 10(1), 17–26.
<https://doi.org/10.1109/TLT.2016.2616312>
- Crompton, H., Burke, D., & Gregory, K. (2017). The use of smartphones in learning in higher education: A literature review. *Journal of Higher Education Policy and Management*, 39(3), 288–300.
<https://doi.org/10.1080/1360080X.2017.1300829>
- Ekubo, E. A., & Esiefarienrhe, B. M. (2022). Using machine learning to predict low academic performance at a Nigerian university. *The African Journal of Information and Communication*, 30, 1–26.
<https://doi.org/10.23962/ajic.i30.14839>
- Ferguson, R. (2012). Learning analytics: Drivers, developments and challenges. *International Journal of Technology Enhanced Learning*, 4(5–6), 304–317.
<https://doi.org/10.1504/IJTEL.2012.051816>
- Google LLC. (2024a). Flutter: Build apps for any screen. Retrieved from <https://flutter.dev>
- Google LLC. (2024b). ML Kit: On-device machine learning. Retrieved from <https://developers.google.com/ml-kit>
- Hipp, D. R. (2024). SQLite. Retrieved from <https://sqlite.org>
- Kaliisa, R., & Michelle, P. (2019). Mobile learning policy and practice in Africa: Towards inclusive and equitable access to higher education. *Australasian Journal of Educational Technology*, 35(6), 1–14.
<https://doi.org/10.14742/ajet.5562>
- Kasneci, E., Seßler, K., Küchemann, S., Bannert, M., Dementieva, D., Fischer, F., . . . Kasneci, G. (2023). ChatGPT for good? On opportunities and challenges of large language models for education. *Learning and Individual Differences*, 103, 102274. <https://doi.org/10.1016/j.lindif.2023.102274>

- Khamis, A. H., Khalil, M. K., Al-Maaitah, A. A., & Al-Maaitah, T. (2023). Mobile application for CGPA calculation based on students academic performance. *International Journal of Evaluation and Research in Education*, 12(3), 1550–1559. <https://doi.org/10.11591/ijere.v12i3.25003>
- Liu, Z., Agrawal, P., Singhal, S., Madaan, V., Kumar, M., & Verma, P. K. (2025). LPITutor: An LLM-based personalized intelligent tutoring system using RAG and prompt engineering. *PeerJ Computer Science*, 11, e2991. <https://doi.org/10.7717/peerj-cs.2991>
- National Universities Commission. (2021). Minimum academic standards for undergraduate programmes in Nigerian universities. Retrieved from <https://www.nuc.edu.ng>
- Nosseir, A., & Fathy, M. (2020). Performance management mobile application for university students. *International Journal of Interactive Mobile Technologies*, 14(2), 4–18. <https://doi.org/10.3991/ijim.v14i02.10940>
- OpenAI. (2023). GPT-4 technical report. Retrieved from <https://arxiv.org/abs/2303.08774>
- Oyefara, J. L., Adejoh, P. E., Adisa, W. B., Abdulsalam, K. A., & Alabi, T. A. (2022). ICT utilisation and associated barriers in teaching among middle-level academics in Nigerian universities. *Journal of Higher Education in Africa*, 20(1), 175–198.
- PostgreSQL Global Development Group. (2023). PostgreSQL 16 documentation. Retrieved from <https://www.postgresql.org/docs/16/>
- Puryanto, J. A., & Akbar, H. (2025). Performance analysis of Provider and Riverpod state management library on Flutter applications. *Journal of Technology and Informatics*, 7(2), 190–200. <https://doi.org/10.37802/joti.v7i2.1164>
- Romero, C., & Ventura, S. (2010). Educational data mining: A review of the state of the art. *IEEE Transactions on Systems, Man, and Cybernetics, Part C: Applications and Reviews*, 40(6), 601–618. <https://doi.org/10.1109/TSMCC.2010.2053532>
- Rousselet, R. (2021). Riverpod: A better state management solution for Flutter. Retrieved from <https://riverpod.dev>

- Siemens, G. (2013). Learning analytics: The emergence of a discipline. *American Behavioral Scientist*, 57(10), 1380–1400.
<https://doi.org/10.1177/0002764213498851>
- Supabase Inc. (2024). Supabase documentation: The open source Firebase alternative. Retrieved from <https://supabase.com/docs>
- Susnjak, T., Ramaswami, G. S., & Mathrani, A. (2022). Learning analytics dashboard: A tool for providing actionable insights to learners. *International Journal of Educational Technology in Higher Education*, 19(1), 1–23.
<https://doi.org/10.1186/s41239-021-00313-7>
- Wang, L., Chen, X., Deng, X., Wen, H., You, M., Liu, W., . . . Li, J. (2023). Large language models in education: Vision and opportunities. In *Proceedings of the 2023 IEEE International Conference on Big Data* (pp. 4842–4851). IEEE.
<https://doi.org/10.1109/BigData59044.2023.10386291>
- Yan, L., Sha, L., Zhao, L., Li, Y., Martinez-Maldonado, R., Chen, G., . . . Gašević, D. (2024). Practical and ethical challenges of large language models in education: A systematic scoping review. *British Journal of Educational Technology*, 55(2), 584–605. <https://doi.org/10.1111/bjet.13370>
- Zawacki-Richter, O., Marín, M. A., Bond, W., & Gouverneur, F. (2019). Systematic review of research on artificial intelligence applications in higher education – where are the educators? *International Journal of Educational Technology in Higher Education*, 16(1), 1–27. <https://doi.org/10.1186/s41239-019-0171-0>
- Zou, D., Zou, X., Huang, Y., & Li, P. (2024). A comparative study on cross-platform mobile application development frameworks: Flutter and React Native. *Malaysian Journal of Computing*, 9(2), 3310–3322.
<https://doi.org/10.24191/mjoc.v9i2.26484>

APPENDICES

Appendix A: CGPA Computation Engine Source Code

The complete source of lib/domain/engine/cgpa_engine.dart, the pure Dart module described in section 4.4, is reproduced below in full, exactly as implemented in the submitted system.

```
/// The CGPA engine.
///
/// ARCHITECTURAL CONTRACT – do not violate:
/// * This file is pure. No I/O, no async, no Flutter imports, no network.
/// * The AI layer NEVER performs arithmetic. It consumes the outputs of
///   this engine via function calling and only phrases them in prose.
///   If a model is asked to compute a CGPA it will hallucinate one.
/// * Every number the user sees must originate here.
library;

import '../models/course_result.dart';
import '../models/grading_scheme.dart';

/// The computed outcome for a single semester.
class SemesterComputation {
  final String semesterId;
  final int level;
  final SemesterTerm term;
  final String session;

  /// Quality points earned this semester (grade point x credit unit, summed).
  final double qualityPoints;

  /// Credit units counted in the denominator for this semester.
  final int creditUnits;

  /// Credit units actually passed (excludes failures).
  final int creditsPassed;

  final double gpa;

  /// Results excluded from the calculation and why.
  final List<ExcludedResult> excluded;

  const SemesterComputation({
    required this.semesterId,
    required this.level,
    required this.term,
    required this.session,
    required this.qualityPoints,
    required this.creditUnits,
    required this.creditsPassed,
    required this.gpa,
    this.excluded = const [],
  });

  String get shortLabel => '${level}L ${term.shortLabel}';
}
```

```

/// A result that was not counted, with a human-readable reason.
/// Surfaced in the UI so a student can always answer "why isn't this
/// course in my CGPA?" without contacting support.
class ExcludedResult {
    final String resultId;
    final String courseCode;
    final String reason;

    const ExcludedResult({
        required this.resultId,
        required this.courseCode,
        required this.reason,
    });
}

/// A problem found during calculation. Never thrown – collected and returned
/// so a single bad row cannot blank the entire dashboard.
class CalculationIssue {
    final String? resultId;
    final String courseCode;
    final String message;
    final IssueSeverity severity;

    const CalculationIssue({
        this.resultId,
        required this.courseCode,
        required this.message,
        this.severity = IssueSeverity.warning,
    });
}

enum IssueSeverity { warning, error }

/// The complete academic standing of a student at a point in time.
class AcademicStanding {
    final double cgpa;
    final double totalQualityPoints;
    final int totalCreditUnits;
    final int totalCreditsPassed;
    final ClassificationBand? classification;
    final List<SemesterComputation> semesters;
    final List<CalculationIssue> issues;

    const AcademicStanding({
        required this.cgpa,
        required this.totalQualityPoints,
        required this.totalCreditUnits,
        required this.totalCreditsPassed,
        this.classification,
        this.semesters = const [],
        this.issues = const [],
    });

    bool get hasData => totalCreditUnits > 0;
    bool get hasErrors =>
        issues.any((i) => i.severity == IssueSeverity.error);

    /// Best semester GPA on record. The projection solver uses this as the
    /// honest benchmark for feasibility rather than inventing a probability.
    double? get bestSemesterGpa {
        final withData = semesters.where((s) => s.creditUnits > 0);
        if (withData.isEmpty) return null;
        return withData.map((s) => s.gpa).reduce((a, b) => a > b ? a : b);
    }
}

```

```

double? get worstSemesterGpa {
    final withData = semesters.where((s) => s.creditUnits > 0);
    if (withData.isEmpty) return null;
    return withData.map((s) => s.gpa).reduce((a, b) => a < b ? a : b);
}

/// Cumulative CGPA after each semester, in chronological order – the
/// series a trend chart plots. A single point cannot show a trend, so
/// callers with fewer than two semesters should not render it as a line.
List<double> get cumulativeCgpaTrend {
    double qp = 0;
    int units = 0;
    return semesters.map((s) {
        qp += s.qualityPoints;
        units += s.creditUnits;
        return units == 0 ? 0.0 : CgpaEngine.round2(qp / units);
    }).toList();
}

/// Trend across the last [window] semesters. Positive means improving.
double? recentTrend({int window = 3}) {
    final withData =
        semesters.where((s) => s.creditUnits > 0).toList();
    if (withData.length < 2) return null;
    final slice = withData.length <= window
        ? withData
        : withData.sublist(withData.length - window);
    return slice.last.gpa - slice.first.gpa;
}

static const empty = AcademicStanding(
    cgpa: 0,
    totalQualityPoints: 0,
    totalCreditUnits: 0,
    totalCreditsPassed: 0,
);

}

/// Pure calculation engine. All methods are static and side-effect free.
class CgpaEngine {
    const CgpaEngine._();

    /// Rounding used for all displayed values. Two decimal places matches
    /// what Nigerian universities print on transcripts.
    static double round2(double value) => (value * 100).roundToDouble() / 100;

    /// Compute one semester's GPA.
    ///
    /// [allResults] is every result across the whole academic record (every
    /// semester, not just earlier ones), needed to detect which failing rows
    /// in this semester are later superseded by a repeat – the repeat is
    /// almost always in a *later* semester, so "earlier only" would never
    /// find it.
    static SemesterComputation computeSemester({
        required Semester semester,
        required GradingScheme scheme,
        List<CourseResult> allResults = const [],
        List<CalculationIssue>? issueSink,
    }) {
        double qualityPoints = 0;
        int creditUnits = 0;
        int creditsPassed = 0;
        final excluded = <ExcludedResult>[];

        for (final result in semester.results) {
            final definition = scheme.definitionForLetter(result.grade);

```

```

// Unknown letter grade. Excluded rather than silently zeroed -
// a silent zero corrupts the CGPA with no visible cause.
if (definition == null) {
    issueSink?.add(CalculationIssue(
        resultId: result.id,
        courseCode: result.courseCode,
        message:
            'Grade "${result.grade}" is not defined in ${scheme.name}.',
        severity: IssueSeverity.error,
    ));
    excluded.add(ExcludedResult(
        resultId: result.id,
        courseCode: result.courseCode,
        reason: 'Unrecognised grade',
    ));
    continue;
}

if (result.creditUnit <= 0) {
    issueSink?.add(CalculationIssue(
        resultId: result.id,
        courseCode: result.courseCode,
        message: 'Credit unit must be greater than zero.',
        severity: IssueSeverity.error,
    ));
    excluded.add(ExcludedResult(
        resultId: result.id,
        courseCode: result.courseCode,
        reason: 'Invalid credit unit',
    ));
    continue;
}

var effectivePoint = definition.point;

// Carryover handling. This is where institutions diverge and where
// a wrong assumption produces a plausible-looking wrong number.
if (result.isRepeat) {
    switch (scheme.repeatPolicy) {
        case RepeatPolicy.countBothAttempts:
            // Nothing to adjust here – the original failure stays in its
            // own semester and this attempt counts normally.
            break;

        case RepeatPolicy.replaceOriginal:
            // The original is excluded when its own semester is computed;
            // see [_isSupersededBy]. This attempt counts at face value.
            break;

        case RepeatPolicy.replaceWithCap:
            final cap = scheme.repeatCapPoint;
            if (cap != null && effectivePoint > cap) {
                effectivePoint = cap;
                issueSink?.add(CalculationIssue(
                    resultId: result.id,
                    courseCode: result.courseCode,
                    message:
                        'Repeat grade capped at $cap under ${scheme.name} policy.',
                    severity: IssueSeverity.warning,
                ));
            }
            break;
    }
}

// Under replaceOriginal, a failure that was later repeated is dropped
// from its original semester.

```

```

        if (scheme.repeatPolicy == RepeatPolicy.replaceOriginal &&
            definition.isFailing &&
            !_isSupersededLater(result, allResults)) {
            excluded.add(ExcludedResult(
                resultId: result.id,
                courseCode: result.courseCode,
                reason: 'Superseded by a later repeat',
            ));
            continue;
        }

        qualityPoints += effectivePoint * result.creditUnit;
        creditUnits += result.creditUnit;
        if (!definition.isFailing) creditsPassed += result.creditUnit;
    }

    final gpa = creditUnits == 0 ? 0.0 : round2(qualityPoints / creditUnits);

    return SemesterComputation(
        semesterId: semester.id,
        level: semester.level,
        term: semester.term,
        session: semester.session,
        qualityPoints: qualityPoints,
        creditUnits: creditUnits,
        creditsPassed: creditsPassed,
        gpa: gpa,
        excluded: excluded,
    );
}

/// Compute full academic standing across all semesters.
///
/// Semesters are sorted chronologically before computation because
/// carryover detection depends on knowing what came before.
static AcademicStanding computeStanding({
    required List<Semester> semesters,
    required GradingScheme scheme,
}) {
    if (semesters.isEmpty) return AcademicStanding.empty;

    final ordered = [...semesters]
        ..sort((a, b) => a.sortKey.compareTo(b.sortKey));

    final allResults = List<CourseResult>.unmodifiable(
        ordered.expand((s) => s.results),
    );

    final issues = <CalculationIssue>[];
    final computations = <SemesterComputation>[];

    double totalQp = 0;
    int totalUnits = 0;
    int totalPassed = 0;

    for (final semester in ordered) {
        final computation = computeSemester(
            semester: semester,
            scheme: scheme,
            allResults: allResults,
            issueSink: issues,
        );

        computations.add(computation);
        totalQp += computation.qualityPoints;
        totalUnits += computation.creditUnits;
    }
}

```

```

        totalPassed += computation.creditsPassed;
    }

    final cgpa = totalUnits == 0 ? 0.0 : round2(totalQp / totalUnits);

    return AcademicStanding(
        cgpa: cgpa,
        totalQualityPoints: totalQp,
        totalCreditUnits: totalUnits,
        totalCreditsPassed: totalPassed,
        classification: scheme.classify(cgpa),
        semesters: computations,
        issues: issues,
    );
}

/// Recalculate from a mutated semester list.
///
/// Editing a 200-level grade must cascade through every downstream
/// semester. Rather than patching incrementally – which drifts – the
/// engine is cheap enough to simply recompute the whole record.
/// Returns both the new standing and the delta for the UI to show.
static RecalculationResult recalculate({
    required List<Semester> semesters,
    required GradingScheme scheme,
    required AcademicStanding previous,
}) {
    final updated = computeStanding(semesters: semesters, scheme: scheme);
    return RecalculationResult(
        previous: previous,
        updated: updated,
        cgpaDelta: round2(updated.cgpa - previous.cgpa),
    );
}

/// Detect whether a failed result is repeated in a later attempt,
/// regardless of whether that attempt falls in an earlier or later
/// semester in the sorted order.
static bool _isSupersededLater(
    CourseResult result,
    List<CourseResult> allResults,
) {
    final code = result.courseCode.trim().toUpperCase();
    return allResults.any((r) =>
        r.id != result.id &&
        r.courseCode.trim().toUpperCase() == code &&
        r.attempt > result.attempt);
}

/// The before/after pair shown to a student when their record changes.
/// Making the change visible is what turns result entry into a reward
/// rather than a chore.
class RecalculationResult {
    final AcademicStanding previous;
    final AcademicStanding updated;
    final double cgpaDelta;

    const RecalculationResult({
        required this.previous,
        required this.updated,
        required this.cgpaDelta,
    });

    bool get improved => cgpaDelta > 0;
    bool get unchanged => cgpaDelta == 0;
}

```

```
bool get classificationChanged =>  
    previous.classification?.label != updated.classification?.label;  
}
```

Appendix B: Sample Unit Test Cases

The following excerpt from `test/engine/cgpa_engine_test.dart` shows a representative sample of the automated tests described in section 4.8, covering grading-scheme integrity, semester GPA computation, and the handling of a failing grade.

```
import 'package:flutter_test/flutter_test.dart';
import 'package:perform_plus/data/seed/nigerian_institutions.dart';
import 'package:perform_plus/domain/engine/cgpa_engine.dart';
import 'package:perform_plus/domain/engine/projection_solver.dart';
import 'package:perform_plus/domain/models/course_result.dart';
import 'package:perform_plus/domain/models/grading_scheme.dart';
```

```
final _now = DateTime(2026, 1, 1);
```

```
CourseResult _r(
  String code,
  int unit,
  String grade, {
  String semesterId = 's1',
  int attempt = 1,
  String? id,
}) =>
  CourseResult(
    id: id ?? '$semesterId-$code-$attempt',
    semesterId: semesterId,
    courseCode: code,
    creditUnit: unit,
    grade: grade,
    attempt: attempt,
    createdAt: _now,
    updatedAt: _now,
  );
```

```
Semester _s(
  String id,
  int level,
  SemesterTerm term,
  List<CourseResult> results,
) =>
  Semester(
    id: id,
    profileId: 'p1',
    session: '2024/2025',
    term: term,
    level: level,
    results: results,
    createdAt: _now,
    updatedAt: _now,
  );
```

```
void main() {
  final scheme = defaultSchemes['futo']!;

  group('scheme integrity', () {
    test('all seeded schemes validate', () {
      for (final entry in defaultSchemes.entries) {
        expect(
          entry.value.validate(),
          isEmpty,
          reason: 'Scheme "${entry.key}" failed validation',
        );
      }
    });
  });
}
```

```

    });
  });

test('classification bands cover the boundaries correctly', () {
  expect(scheme.classify(5.00)?.shortLabel, 'First Class');
  expect(scheme.classify(4.50)?.shortLabel, 'First Class');
  expect(scheme.classify(4.49)?.shortLabel, '2:1');
  expect(scheme.classify(3.50)?.shortLabel, '2:1');
  expect(scheme.classify(3.49)?.shortLabel, '2:2');
  expect(scheme.classify(2.40)?.shortLabel, '2:2');
  expect(scheme.classify(2.39)?.shortLabel, 'Third Class');
});

test('score maps to the correct letter', () {
  expect(scheme.letterForScore(85), 'A');
  expect(scheme.letterForScore(70), 'A');
  expect(scheme.letterForScore(69), 'B');
  expect(scheme.letterForScore(39), 'F');
});

group('semester GPA', () {
  test('computes a straightforward semester', () {
    // 5*3 + 4*3 + 5*2 + 3*2 + 5*2 = 15+12+10+6+10 = 53 over 12 units
    final semester = _s('s1', 300, SemesterTerm.first, [
      _r('CSC301', 3, 'A'),
      _r('MTH301', 3, 'B'),
      _r('PHY301', 2, 'A'),
      _r('GST301', 2, 'C'),
      _r('CSC303', 2, 'A'),
    ]);

    final result = CgpaEngine.computeSemester(
      semester: semester,
      scheme: scheme,
    );

    expect(result.creditUnits, 12);
    expect(result.qualityPoints, 53);
    expect(result.gpa, closeTo(4.42, 0.01));
  });

  test('a failing grade counts in the denominator but adds no points', () {
    final semester = _s('s1', 100, SemesterTerm.first, [
      _r('CSC101', 3, 'A'),
      _r('MTH101', 3, 'F'),
    ]);

    final result =
      CgpaEngine.computeSemester(semester: semester, scheme: scheme);

    expect(result.creditUnits, 6);
    expect(result.creditsPassed, 3);
    expect(result.qualityPoints, 15);
    expect(result.gpa, 2.50);
  });

  test('an empty semester yields zero, not a division error', () {
    final semester = _s('s1', 100, SemesterTerm.first, []);
    final result =
      CgpaEngine.computeSemester(semester: semester, scheme: scheme);

    expect(result.gpa, 0);
    expect(result.creditUnits, 0);
  });
});

```


Appendix C: Repeat Policy — Worked Numerical Example

The following worked example, used throughout Chapters Four and Five, illustrates how an identical transcript resolves to three different CGPA outcomes depending on the institution's repeat-course policy.

Transcript: a 3-unit course failed with grade F (0.0 points) in Semester 1, then retaken and passed with grade A (5.0 points) in Semester 2, alongside a further 6 units of other results averaging 3.0 grade points in Semester 1.

Table C.1: Repeat Policy — Worked Numerical Example

Repeat Policy	Resulting CGPA	Credit Units Counted
countBothAttempts	3.00	9
replaceOriginal	4.50	6
replaceWithCap (cap = 3.0)	2.67	9

Source: Derived from CgpaEngine.computeStanding, Researcher, 2025